

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Уральский государственный экономический университет»
(УрГЭУ)

КУРСОВАЯ РАБОТА

на тему: Разработка информационной системы для подбора косметических средств

по междисциплинарному курсу МДК.02.01 Технология разработки программного обеспечения

МДК (номер и название)

профессионального модуля ПМ 02 ЭК Экзамен по модулю (Осуществление интеграции программных модулей)

ПМ. (номер и название)

Колледж УрГЭУ

Специальность 09.02.07 Информационные системы и программирование

Студент Устюжанина Ю.В.

Группа 23-03 ИСиП

Дата защиты: 21.10.2026

Руководитель Коломейцева В.С., преподаватель

Оценка: _____

(ФИО, должность, звание)

Екатеринбург

2026

СОДЕРЖАНИЕ

Введение 4
1 Анализ предметной области 6
1.1 анализ объекта исследования 6
1.1.1 Общая характеристика процесса выбора косметики 6
1.1.2 Характеристика субъектов и объектов системы 6
1.1.3 Основные проблемы и недостатки текущего процесса 7
1.2 Стратегия информатизации объекта исследования 7
1.2.1 Обзор существующих решений 7
1.2.2 Цель и принципы стратегий 8
1.2.3 Механизм реализации стратегии 9
1.3 Анализ предмета исследования 9
1.3.1 Суть алгоритма подбора косметических средств 9
1.3.2 Методика расчёта рейтинга совместимости 10
1.3.3 Моделирование пользовательских сценариев 11
1.4 Постановка задачи на автоматизацию 12
1.4.1 Функциональные требования к системе 12
1.4.2 Нефункциональные требования 13
2 Проектная часть 14
2.1 Проектирование информационного обеспечения 14
2.1.1 Архитектура системы 14
2.1.2 Обоснование выбора СУБД 16
2.1.3 Логическая модель базы данных 17
2.1.4 Физическая реализация базы данных 18
2.1.5 Классификаторы и система кодирования информации 19
2.1.6 Организация информационной базы 21
2.1.7 Формы выходных документов 22
2.2 Проектирование программного обеспечения 24
2.2.1 Структура программного обеспечения 24
2.2.2 Алгоритм подбора – ядро системы 29

2.2.3 Логика обработки ответов пользователя 31
2.2.4 Моделирование пользовательских сценариев 31
2.3 Управление проектом 34
2.3.1 Календарный план-график работ 34
2.3.2 Бюджет на разработку 35
2.3.3 Оценка рисков и способы их минимизации 35
3 Разработка информационной системы 36
3.1 Программно-техническая реализация решения поставленной задачи 36
3.1.1 Структура программного обеспечения 36
3.1.2 Спецификации программных модулей 36
3.1.3 Пользовательский интерфейс 37
3.1.4 Обеспечение информационной безопасности 41
3.2 Тестирование, оценка качества и внедрение 43
3.2.1 Исходные данные и результаты работы системы 43
3.2.2 Поэтапное описание работы с системой 45
3.2.3 Порядок внедрения проекта 46
3.3 Оценка эффективности проекта 47
3.3.1 Анализ затрат на ресурсное обеспечение 47
3.3.2 Качественные и количественные эффекты 47
3.3.3 Расчёт экономической эффективности 48
Заключение 49
Список использованных источников 51
Приложение А 53
Приложение Б 54
Приложение В 55
Приложение Г 56
Приложение Д 57
Приложение Е 59

ВВЕДЕНИЕ

Тема моей курсовой работы – разработка информационной системы для подбора косметических средств. Я выбрала её не случайно: сегодня рынок косметики переполнен, на нём представлены сотни брендов и тысячи продуктов с разными составами, обещаниями и ценниками. Разобраться в этом обычному человеку сложно, а профессионального консультанта рядом часто нет. Люди покупают наугад, ориентируясь на рекламу или советы знакомых, и в итоге тратят деньги на то, что им не подходит.

Актуальность темы заключается в том, что существующие решения не обеспечивают полноценной персонализации. Единой системы, которая объединяла бы анализ кожи, бюджет, аллергии, сезонность и образ жизни, на рынке нет.

Цель курсовой работы – разработать информационную систему, которая подбирает косметические средства на основе анкеты пользователя и алгоритма сравнения продуктов.

Задачи исследования:

– изучить, какие параметры влияют на выбор косметики, и собрать их в единую анкету;

– продумать логику подбора: как сравнивать продукты и присваивать им рейтинг для каждого пользователя;

– спроектировать базу данных для хранения пользователей, анкет, продуктов и истории подборок;

– разработать веб-приложение на «C#» с использованием «ASP.NET Core»;

– реализовать ключевые функции: регистрацию, анкету, подбор, личный кабинет, корзину, избранное, оформление заказа и админ-панель;

– настроить «PWA-функционал», чтобы приложение можно было установить на телефон;

– протестировать систему и оценить её экономическую эффективность.

Объект исследования – процесс выбора и подбора косметики для ухода за кожей.

Предмет исследования – алгоритмический процесс подбора косметических продуктов на основе формализованных параметров пользователя, реализованный в виде информационной системы.

При написании работы я использовала анализ научной и аналитической литературы, сравнительный анализ существующих решений, методы обработки данных, алгоритмическое моделирование, а также экспериментальное тестирование системы.

Вопросы персонализированного подбора косметики рассматриваются в научных статьях и отраслевых исследованиях, но готового решения, доступного обычному пользователю, пока не существует.

Научная новизна работы заключается в разработке оригинального алгоритма подбора с весовыми коэффициентами, который учитывает 16 факторов одновременно – от типа кожи и бюджета до уровня стресса и типа питания.

Практическая значимость состоит в том, что система помогает пользователю экономить время и деньги, а магазину – повышать доверие клиентов.

В работе использованы данные о рынке косметики, аналитические обзоры, документация по «ASP.NET Core», «Entity Framework Core» и «PostgreSQL», а также материалы по «PWA» и веб-разработке.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложения. В первой главе анализируется предметная область: рынок косметики, существующие решения, суть алгоритма подбора и требования к системе. Во второй главе описывается проектная часть: архитектура, база данных, алгоритм и управление проектом. В третьей главе представлена реализация системы, её тестирование и оценка эффективности.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 АНАЛИЗ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1.1 Общая характеристика процесса выбора косметики

Объектом моего исследования является процесс выбора и подбора косметических средств для ухода за кожей. Этот процесс сегодня касается миллионов людей: женщины и мужчины, подростки и люди старшего возраста – все они сталкиваются с необходимостью выбирать средства для ежедневного ухода. Однако, несмотря на огромное количество предложений на рынке, сам выбор остаётся сложным, запутанным и часто приводит к разочарованию.

Российский рынок косметики растёт на 5-7% в год. На рынке – сотни брендов и тысячи продуктов: кремы, сыворотки, тоники, маски, пилинги. Для обычного человека это многообразие – проблема: он не знает, какие компоненты ему подходят, а информация о составе написана сложным языком [1, 2, 3].

Степень автоматизации этого процесса сегодня остаётся низкой: маркетплейсы дают только простые фильтры, а сервисы не учитывают комплекс факторов. Единой информационной системы, которая объединяла бы анализ кожи, бюджет, аллергии и сезонность, на рынке нет [4, 5].

1.1.2 Характеристика субъектов и объектов системы

В процессе выбора и покупки косметики я выделяю несколько основных групп участников: покупатели, магазины косметики, а также консультанты и косметологи.

Покупатели нуждаются в уходе за кожей, у них разные типы кожи, проблемы и бюджеты, но большинство не имеют профессиональных знаний и не могут самостоятельно определить, какое средство им подойдёт. Магазины косметики – это офлайн-точки продаж и интернет-магазины, которые заинтересованы в том, чтобы покупатель остался доволен и вернулся снова, но часто не могут предложить действительно персонализированную помощь. Консультанты и косметологи помогают подбирать средства, однако их помощь

либо платная, либо зависит от места работы: например, консультант в магазине может рекомендовать только те бренды, которые есть в этом магазине, и не всегда объективен.

Чаще всего выбор происходит: наугад, по совету друзей или знакомых, через консультанта или посредством самостоятельного изучения.

Объектами системы являются: анкета пользователя (15 параметров), продукт (29 характеристик), подборка (10 лучших товаров) и история подборок, сохраняемая в личном кабинете.

1.1.3 Основные проблемы и недостатки текущего процесса

На сегодняшний день процесс выбора косметики остаётся слабо автоматизированным. Маркетплейсы предлагают фильтры по типу кожи и цене, но это лишь поверхностное решение. Ни один из существующих сервисов не предлагает комплексного подхода, учитывающего все факторы одновременно: тип кожи, бюджет, аллергии, сезон, цель ухода, уровень стресса, тип питания и другие параметры [4, 5, 7].

Анализ показал, что текущий уровень автоматизации процесса выбора косметики – низкий. Это подтверждает необходимость разработки собственной информационной системы.

1.2 СТРАТЕГИЯ ИНФОРМАТИЗАЦИИ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.2.1 Обзор существующих решений

Сегодняшние решения для подбора косметики можно разделить на три типа. Первый – фильтры на маркетплейсах, второй – приложения с тестами, третий – профессиональные консультации.

Фильтры на маркетплейсах – это самый распространённый, но и самый примитивный способ: пользователь выбирает категорию, ценовой диапазон и тип кожи, а система показывает все товары, которые подходят хотя бы под один из этих параметров. Никакого комплексного анализа не происходит – если у меня сухая кожа, бюджет 3000 рублей и аллергия на отдушки, я всё равно буду просматривать сотни кремов. Приложения с тестами предлагают пройти опрос и получить рекомендации, что уже шаг вперёд. Однако их алгоритмы закрыты:

пользователь не видит, почему ему рекомендовали именно этот продукт. Нет возможности сохранять историю подборок, нельзя сравнивать продукты или возвращаться к старым рекомендациям. Кроме того, такие приложения часто не учитывают сезонность и аллергии. Профессиональные консультации – самый точный, но и самый дорогой способ: от 2000 рублей и выше.

Анализ конкурентов показал, что ни один из существующих сервисов не предлагает одновременно прозрачного алгоритма подбора, комплексного учёта 15+ параметров и сохранения истории рекомендаций. Это подтверждает актуальность разработки системы [4, 5, 7].

1.2.2 Цель и принципы стратегий

Целевая аудитория – покупатели косметики 18–50 лет, преимущественно девушки. Они хотят быстрого и точного подбора без консультантов, а магазины – роста лояльности и снижения возвратов [8].

Исходя из анализа существующих решений, я формулирую стратегию информатизации прикладных процессов:

Цель стратегии: создать открытую, прозрачную и доступную систему, которая позволит пользователю самостоятельно, быстро и без лишних затрат подобрать косметические средства с учётом его особенностей.

Основные принципы стратегии – комплексность, прозрачность, доступность, сохранение истории и экономическая целесообразность.

Комплексность означает, что система учитывает не один, а сразу 15 параметров пользователя: тип кожи, возраст, бюджет, проблемы, аллергии, сезон, любимые бренды, цель ухода, уровень стресса, тип питания, чувствительность к солнцу, склонность к отёкам, профессиональный уход, предпочтение по текстуре и готовность к многоступенчатому уходу. Это позволяет максимально точно подобрать средства. Прозрачность – пользователь видит, как работает алгоритм: какие параметры учитывались и почему конкретному продукту был присвоен тот или иной рейтинг. Доступность – система реализована как веб-приложение с поддержкой «PWA»,

то есть она работает на любом устройстве: компьютере, планшете, смартфоне, а установка на телефон не требует «App Store» или «Google Play».

Сохранение истории: все подборки автоматически сохраняются в личном кабинете пользователя, и он может вернуться к ним через месяц, сезон или год, чтобы отслеживать изменения и корректировать уход. Экономическая целесообразность – система помогает экономить деньги, не покупая неподходящие средства, и время, сокращая процесс подбора с 3-4 часов до 5 минут.

1.2.3 Механизм реализации стратегии

Для реализации этой стратегии:

– разрабатываю базу данных на «PostgreSQL»;

– создаю алгоритм на «C#» с весовыми коэффициентами;

– проектирую веб-приложение на «ASP.NET Core»;

– настраиваю «PWA-функционал»;

– тестирую систему и доказываю её эффективность.

1.3 АНАЛИЗ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.3.1 Суть алгоритма подбора косметических средств

Предметом моего исследования является алгоритмический процесс подбора косметических средств на основе формализованных параметров пользователя. Это процесс, который я автоматизирую: вместо того чтобы человек вручную сравнивал сотни продуктов, система делает это за него – быстро, точно и объективно [8].

Алгоритм получает на вход 15 параметров из анкеты пользователя. Условно их можно разделить на три группы: характеристики кожи, финансовые и сезонные предпочтения, а также дополнительные факторы.

К характеристикам кожи относятся: тип кожи (сухая, жирная, комбинированная, нормальная, чувствительная, обезвоженная, склонная к куперозу, не знаю); возраст (до 20, 20-30, 30-40, 40-50, 50+); проблемы (акне, пигментация, морщины, купероз, тусклый цвет, гиперчувствительность,

чёрные точки, сухость); аллергии (парабены, отдушки, спирт, силиконы, сульфаты, глютен, орехи, эфирные масла); чувствительность к солнцу (да/нет); склонность к отёкам (да/нет); профессиональный уход (да/нет); предпочтение по текстуре (лёгкая/плотная).

К финансовым и сезонным предпочтениям относятся: бюджет (до 1000, 1000-3000, 3000-10000, без лимита); сезон (лето, зима, демисезон); любимые бренды; цель ухода (увлажнение, питание, антивозрастной, очищение, защита, восстановление, матирование, лифтинг, осветление пигментации, сужение пор, сияние, снятие стресса). К дополнительным факторам относятся: уровень стресса (низкий, средний, высокий); тип питания (сбалансированное, сладкое/жирное, вегетарианство); готовность к многоступенчатому уходу (да/нет).

Для каждого продукта, загруженного администратором в каталог, алгоритм вычисляет рейтинг совместимости.

Каждое совпадение добавляет баллы, несовпадение – штраф. Чем выше «Score», тем лучше товар подходит пользователю.

1.3.2 Методика расчёта рейтинга совместимости

Рейтинг рассчитывается как сумма баллов по каждому параметру. Я назначаю весовые коэффициенты в зависимости от важности параметра: например, совпадение типа кожи даёт больше баллов, чем совпадение сезона. Конкретные значения коэффициентов я подбираю на этапе разработки, но общий принцип выглядит так: рейтинг рассчитывается как сумма баллов по 16 факторам – совпадение типа кожи, попадание в бюджет, совпадение проблемы, учёт аллергий, сезон, любимые бренды, цель ухода, возраст, уровень стресса, тип питания, чувствительность к солнцу, склонность к отёкам, текстура, готовность к многоступенчатому уходу, профессиональный уход и бонусы. Чем больше итоговая сумма, тем лучше продукт подходит пользователю. Продукты сортируются по убыванию рейтинга, и пользователь видит подборку рекомендаций.

Веса подобраны эмпирически: важные параметры (тип кожи, проблема, бюджет) дают +20…+40 баллов, второстепенные (текстура, сезон) – +5…+15. Штраф за аллергены – до -50 [13].

1.3.3 Моделирование пользовательских сценариев

Я моделирую несколько основных сценариев работы системы. Сначала новый пользователь заходит на сайт, видит главную страницу с приветствием и кнопкой «Начать подбор», нажимает на неё и попадает в анкету. Затем он заполняет анкету из 15 шагов: вопросы появляются по одному, есть прогресс-бар, кнопка «Продолжить» переключает шаги. На последнем шаге кнопка меняется на «Получить подборку».

После этого пользователь получает подборку: алгоритм обрабатывает данные анкеты и формирует список продуктов с рейтингом, ценой, фото и описанием, а пользователь видит, почему каждый продукт ему подходит. Далее он может добавить понравившиеся товары в корзину, добавить их в избранное или сохранить подборку в личном кабинете, чтобы вернуться к ней позже. Через месяц пользователь открывает личный кабинет, видит историю подборок, может обновить анкету и получить новые рекомендации.

Все сценарии визуализированы в нотации «BPMN» (бизнес-процессная модель), которая показывает взаимодействие пользователя и системы на каждом этапе. Это позволяет увидеть, какие действия выполняет человек, а какие – автоматика.

На сегодняшний день все эти действия выполняются вручную и разрозненно. Человек сам собирает информацию о своей коже, сам ищет продукты, сам сравнивает, сам запоминает или теряет результаты. Это неэффективно и дорого.

Основные проблемы текущего процесса:

– невозможность одновременно учесть все параметры выбора вручную;

– большая трудоёмкость;

– низкая достоверность результатов;

– отсутствие систематизации и истории;

– зависимость от субъективного мнения консультантов или случайных советов.

Я предлагаю перевести этот процесс в автоматизированную систему: она будет собирать данные, анализировать, сравнивать и сохранять историю.

1.4 ПОСТАНОВКА ЗАДАЧИ НА АВТОМАТИЗАЦИЮ

1.4.1 Функциональные требования к системе

Цель автоматизации – создать информационную систему, которая позволяет пользователю подбирать косметические средства на основе анкеты и алгоритма сравнения продуктов.

Проектируемый комплекс задач является частью информационной системы и включает в себя следующие функциональные блоки:

– управление пользователями (регистрация, авторизация, личный кабинет);

– сбор и хранение данных анкеты;

– алгоритмический расчёт рейтинга продуктов;

– формирование подборки рекомендаций;

– управление каталогом продуктов (админ-панель);

– работа с корзиной и избранным;

– поддержка PWA для мобильных устройств.

В основе системы лежит пошаговая анкета, после заполнения которой запускается алгоритм подбора. Всё взаимодействие строится по схеме: Пользователь – Анкета – Алгоритм – Подборка – Корзина/Избранное/История.

Я использую нотацию «BPMN» для описания процессов. Пользователь открывает сайт, заполняет анкету и нажимает «Получить подборку». Алгоритм обрабатывает данные, и система выводит подборку продуктов. После этого пользователь может добавить товары в корзину или избранное либо сохранить подборку.

Каждая функция системы описана через входные данные, выходные данные и регламент выполнения. Спецификация функциональных требований приведена в таблице 1.

Таблица 1 – Спецификация функциональных требований[1]

Функция	Вход	Выход	Регламент
Регистрация	Имя, «email», пароль	Созданный аккаунт	Ввод данных – проверка на уникальность – сохранение в БД
Заполнение анкеты	Параметры	Сохранённый профиль	Пошаговый опрос – сохранение данных в БД
Личный кабинет	ID пользователя	Данные профиля, история подборок, избранное	Запрос к БД – отображение на странице
Подбор продуктов	Данные анкеты	Подборка продуктов с рейтингом	Алгоритм сравнивает анкету с каждым продуктом – вычисляет рейтинг – сортирует – выводит результат
Корзина	Список выбранных продуктов	Итоговая стоимость	Добавление/удаление – пересчёт суммы
Админ-панель	Данные о продукте	Добавление/редактирование/удаление продукта	Заполнение полей – сохранение – обновление каталога

1.4.2 Нефункциональные требования

Требования к программно-технической среде: система реализована на «C#» с использованием «ASP.NET Core MVC»; база данных – «PostgreSQL»; среда разработки – «Visual Studio»; интерфейс – «HTML», «CSS», «Bootstrap», «JavaScript»; хостинг – «GitHub Pages» для статики и облачный сервер для бэкенда; протокол – «HTTPS».

Пользовательские требования включают быстродействие (подборка формируется за 1–2 секунды), надёжность (корректная обработка всех входящих данных), информационную безопасность (пароли хранятся в зашифрованном виде, используется «HTTPS») и эргономику (интерфейс интуитивно понятен и не требует специальной подготовки).

[1] Составлено автором

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ

2.1.1 Архитектура системы

Разрабатываемая система представляет собой веб-приложение на «ASP.NET Core MVC» с базой данных на «PostgreSQL». Выбор архитектурного паттерна «Model-View-Controller» обусловлен необходимостью чёткого разделения данных, бизнес-логики и отображения. Такое разделение упрощает разработку и поддержку: изменение интерфейса не затрагивает логику, а корректировка алгоритма не требует переписывания представлений [9].

Архитектура системы является классической клиент-серверной. Пользователь открывает сайт в браузере или в режиме «PWA», если приложение установлено на мобильное устройство. Браузер отправляет «HTTP-запрос» на сервер, сервер обрабатывает его через контроллеры, обращается к базе данных через «Entity Framework Core», получает необходимые данные и возвращает готовую страницу. Всю логику подбора я вынесла в отдельный сервис «RecommendationService», чтобы не дублировать код в контроллерах.

Система разделена на три уровня. Клиент – это браузер или «PWA» на мобильном устройстве, он отвечает за отображение данных и отправку запросов. Сервер работает на «ASP.NET Core MVC» и содержит контроллеры для обработки запросов, сервисы с бизнес-логикой и контекст базы данных для работы с данными. База данных – «PostgreSQL», хранит пользователей, анкеты, товары, подборки, заказы и избранное.

Схема взаимодействия уровней системы приведена на рисунке 1.

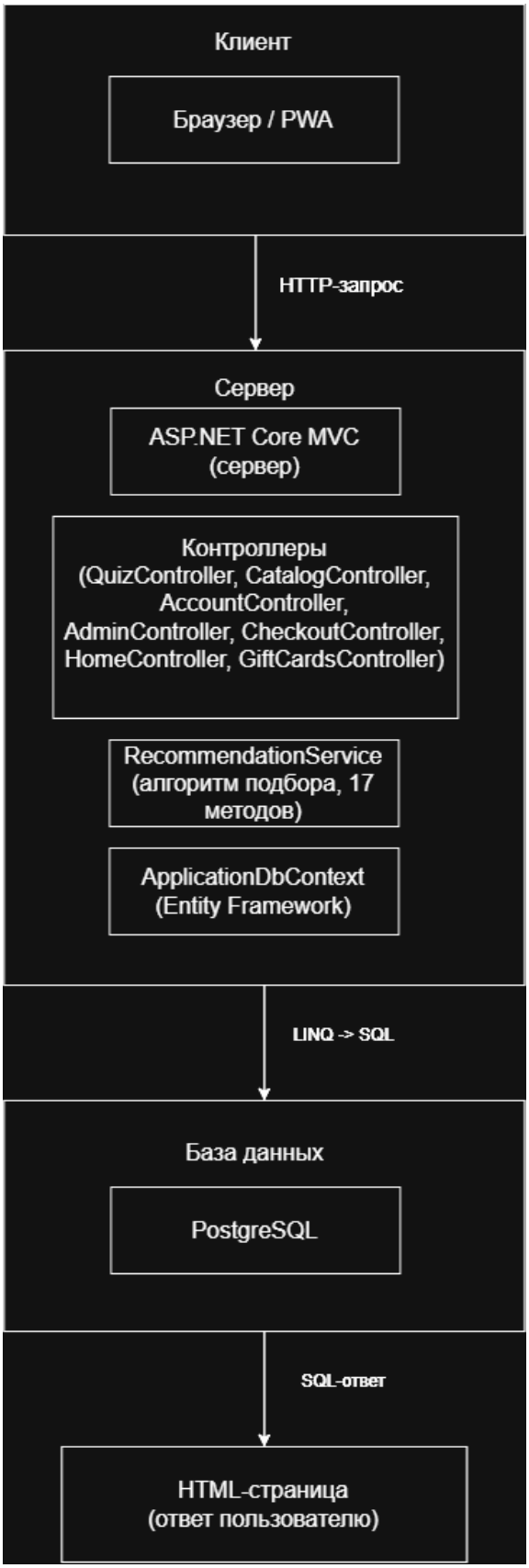


Рисунок 1 – Схема клиент-серверного взаимодействия[2]

[2] Составлено автором

2.1.2 Обоснование выбора СУБД

В качестве системы управления базами данных выбрана «PostgreSQL». Выбор обоснован несколькими критериями: она распространяется бесплатно, что существенно для учебного проекта, и отличается надёжностью: используется в крупных промышленных проектах и демонстрирует высокую устойчивость к нагрузкам. Для интеграции с «.NET» существует официальный провайдер «Npgsql.EntityFrameworkCore.PostgreSQL», подключаемый через «NuGet».

Отдельно стоит отметить поддержку «timestamp with time zone» – это критично для хранения дат подборок, заказов и регистрации, все даты сохраняются в «UTC» для исключения ошибок при смене часовых поясов. Кроме того, «PostgreSQL» работает с массивами и «JSON», что при необходимости позволяет хранить сложные структуры данных [12].

Через «NuGet» установлены четыре ключевых пакета:

– «Npgsql.EntityFrameworkCore.PostgreSQL» (8.0.8): провайдер для «PostgreSQL»;

– «Microsoft.EntityFrameworkCore.Tools» (8.0.8): инструменты миграций;

– «Microsoft.EntityFrameworkCore.Design» (8.0.8): «design-time» компоненты;

– «BCrypt.Net-Next (4.2.0)»: библиотека хеширования паролей.

Базовый пакет «Microsoft.EntityFrameworkCore» подключён автоматически как транзитивная зависимость. Всего установлено 38 транзитивных пакетов. Состав установленных пакетов показан на рисунке 2.

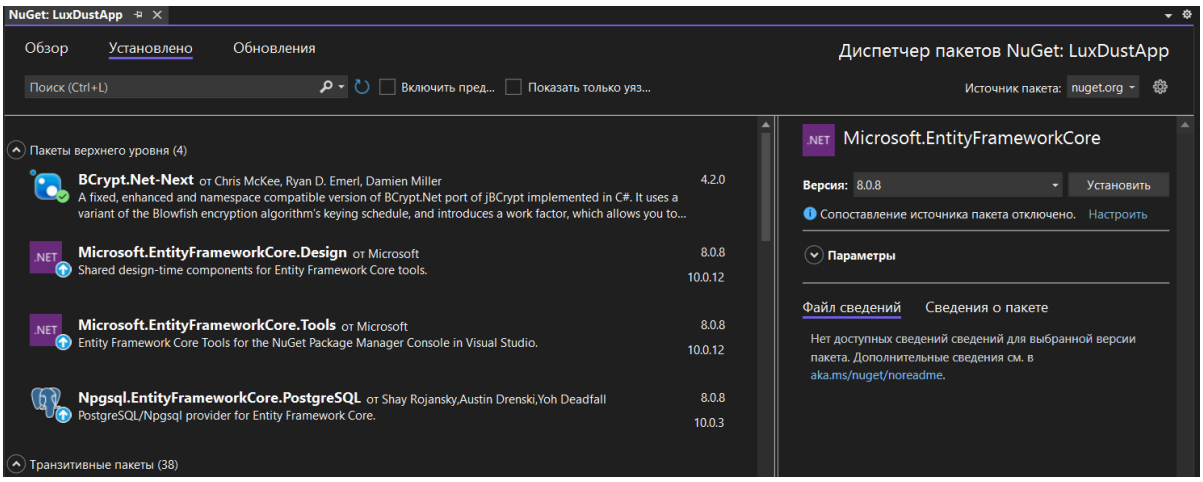


Рисунок 2 – Установленные «NuGet-пакеты»[3]

2.1.3 Логическая модель базы данных

База данных включает 11 таблиц. Краткое описание таблиц приведено в таблице 2.

Таблица 2 – Состав таблиц базы данных[4]

Таблица	Назначение
«Users»	Пользователи: «email», хеш пароля, имя, роль (админ), дата регистрации
«Profiles»	Анкеты: 15 полей (тип кожи, возраст, бюджет, проблемы, аллергии, сезон, бренды, цель, стресс, питание, солнце, отёки, проф. уход, текстура, многоступенчатый уход)
«Products»	Товары: название, бренд, категория, подкатегория, цена, тип кожи, проблема, сезон, описание, рейтинг, флаги «Новинка»/«Акция», 8 аллергенов, дополнительные параметры
«Recommendations»	История подборок: товар, пользователь, рейтинг, дата
«Favorites»	Избранное: пользователь, товар, дата добавления
«Carts»	Корзина: пользователь, товар, количество, дата
«PromoCodes»	Промокоды: код, процент скидки, фиксированная скидка, минимальная сумма, срок действия, лимит использований, счётчик использования
«GiftCards»	Подарочные карты: код, номинал, остаток, активность, владелец, дата покупки, дата использования, срок действия
«OrderGiftCards»	Связь заказов и подарочных карт: заказ, карта, сумма списания
«Orders»	Заказы: способ доставки, адрес, дата, время, телефон, сумма, статус
«OrderItems»	Товары в заказе: заказ, товар, количество, цена на момент заказа

Связи между таблицами реализованы в нотации «Crow's Foot». Класс «User» связан с «Profile» (1:1). «User» и «Product» связаны с

[3] Составлено автором
[4] Составлено автором

«Recommendation», «Favorite», «Cart», «Order (1:N)». Между «Order» и «OrderItem» – обязательная связь «1:N».

Логическая структура базы данных представлена в виде «ER-диаграммы». На ней отражены 11 таблиц, их атрибуты, первичные и внешние ключи, а также связи между сущностями в нотации «Crow's Foot». Полная схема приведена в Приложении А.

2.1.4 Физическая реализация базы данных

Физическая реализация выполнена через миграции «Entity Framework Core». «SQL-скрипты» генерируются автоматически на основе «C#-моделей». При добавлении нового поля в модель я выполняю команду «Add-Migration», затем «Update-Database» – база обновляется без ручного написания «SQL» [10].

Пример SQL-скрипта создания таблицы «Products» представлен в Приложении Б.

Все 29 полей таблицы «Products» проверены через «pgAdmin»: «integer», «text», «Boolean», «double precision», «timestamp with time zone». Поля «Name», «Brand», «Category», «SubCategory», «SkinType», «Problem», «Season», «ImageUrl», «Description», «TexturePreference» – «nullable». Структура таблицы «Products» представлена на рисунке 3.

Data Output | Messages | Notifications

	column_name name	data_type character varying	is_nullable character varying (3)	column_default character varying
1	Id	integer	NO	[null]
2	Name	text	NO	[null]
3	Brand	text	NO	[null]
4	Category	text	NO	[null]
5	Price	integer	NO	[null]
6	SkinType	text	NO	[null]
7	Problem	text	NO	[null]
8	AllergenFree	boolean	NO	[null]
9	Season	text	NO	[null]
10	ImageUrl	text	NO	[null]
11	Description	text	NO	[null]
12	Rating	double precision	NO	[null]
13	CreatedAt	timestamp with time zo...	NO	[null]
14	SunSensitivity	boolean	YES	false
15	TendencyToEdema	boolean	YES	false
16	TexturePreference	text	YES	[null]
17	HasProfessionalC...	boolean	YES	false
18	ReadyForMultiStep	boolean	YES	false
19	SubCategory	text	YES	''::text
20	IsNew	boolean	YES	false
21	IsOnSale	boolean	YES	false
22	HasParabens	boolean	YES	false
23	HasFragrance	boolean	YES	false
24	HasAlcohol	boolean	YES	false
25	HasSilicones	boolean	YES	false
26	HasSulfates	boolean	YES	false
27	HasGluten	boolean	YES	false
28	HasNuts	boolean	YES	false
29	HasEssentialOils	boolean	YES	false

Рисунок 3 – Структура таблицы «Products» в «pgAdmin»[5]

В таблице «Profiles» большинство полей также «nullable», поскольку пользователь может не заполнить часть анкеты – например, не указать любимые бренды или точный тип кожи. Обязательными остаются «Id», «UserId», «Age», «Budget» и булевы поля («SunSensitivity», «TendencyToEdema», «HasProfessionalCare», «ReadyForMultiStep»). В таблице «Users» все поля обязательные: без «email», пароля и имени регистрация невозможна.

2.1.5 Классификаторы и система кодирования информации

В системе используется несколько классификаторов, обеспечивающих единообразие обработки данных. Структура кодовых обозначений объектов показана в таблице 3.

[5] Составлено автором

Таблица 3 – Структура кодовых обозначений объектов.[6]

Наименование кодируемого множества объектов	Значность кода	Система кодирования	Вид классификатора
Тип кожи	1	Порядковая	Локальный
Возрастная группа	1	Порядковая	Локальный
Диапазон бюджета	1	Порядковая	Локальный
Проблема кожи	2	Порядковая	Локальный
Аллерген	2	Порядковая	Локальный
Сезон	1	Порядковая	Локальный
Цель ухода	2	Порядковая	Локальный
Категория товара	2	Порядковая	Локальный
Подкатегория товара	3	Комбинированная	Локальный
Статус заказа	1	Порядковая	Локальный

Все классификаторы являются локальными, поскольку разработаны специально для данной предметной области и не имеют общероссийского или международного аналога. Преобладает порядковая система кодирования: каждому значению присваивается порядковый номер внутри группы. Для подкатегорий используется комбинированная система: первые две цифры означают код родительской категории, третья – номер подкатегории внутри неё.

Полный перечень классификаторов продемонстрирован в таблице 4.

Таблица 4 – Перечень классификаторов и их значений[7]

Классификатор	Значения	Где используется
Тип кожи	Сухая, Жирная, Комбинированная, Нормальная, Чувствительная, Обезвоженная, Склонная к куперозу, Не знаю	«Profiles», «Products»
Возраст	До 20, 20-30, 30-40, 40-50, 50+	«Profiles»
Бюджет	До 1000, 1000-3000, 3000-10000, 10000-25000, Без лимита	«Profiles»
Проблемы	Акне, Пигментация, Морщины, Купероз, Тусклый цвет, Гиперчувствительность, Чёрные точки, Сухость, Расширенные поры, Отёчность и др.	«Profiles», «Products»
Аллергии	Парабены, Отдушки, Спирт, Силиконы, Сульфаты, Глютен, Орехи, Эфирные масла	«Profiles», «Products»
Сезон	Лето, Зима, Демисезон, Круглый год	«Profiles», «Products»

[6] Составлено автором
[7] Составлено автором

Продолжение таблицы 4

Классификатор	Значения	Где используется
Цель ухода	Увлажнение, Питание, Антивозрастной, Очищение, Защита, Восстановление, Матирование, Лифтинг, Осветление, Сужение пор, Сияние, Снятие стресса	«Profiles», «Products»
Уровень стресса	Низкий, Средний, Высокий	«Profiles»
Тип питания	Сбалансированное, Сладкое/жирное, Вегетарианство	«Profiles»
Текстура	Лёгкая, Плотная	«Profiles», «Products»
Категории	17 категорий (Уход за лицом, Макияж, Волосы, Тело, Парфюмерия и др.)	«Products»
Подкатегории	Около 80 подкатегорий	«Products»

Методика ведения НСИ: администратор через админ-панель добавляет новые категории и подкатегории. Новая категория автоматически появляется в каталоге и фильтрах. При удалении товара он исчезает из подборок, но остаётся в истории заказов – это обеспечивает сохранность данных для отчётности.

2.1.6 Организация информационной базы

Информационная база представляет собой базу данных «PostgreSQL» и файловое хранилище изображений товаров. Изображения не хранятся в БД, поскольку это замедляет запросы и увеличивает резервные копии. В поле «ImageUrl» сохраняется путь к файлу (например, «/images/products/collagen-serum.jpg»), сам файл размещается в папке «wwwroot/images/products/».

Все данные делятся на две категории: нормативно-справочная информация и оперативная информация.

Нормативно-справочная информация (НСИ) – это «медленные» данные, которые редко изменяются: классификаторы (тип кожи, проблемы, аллергии, сезоны, цели ухода), категории и подкатегории товаров, справочная информация о брендах. Оперативная информация (ОИ) – это «быстрые» данные, которые изменяются ежедневно: пользователи и их анкеты, подборки, избранное, корзина, заказы и их статусы.

Каждая таблица базы данных имеет своё назначение, тип записи и регламент применения. Описание приведено в таблице 5.

Таблица 5 – Описание таблиц НСИ и ОИ[8]

Таблица	Тип	Назначение	Когда применяется
«Users»	ОИ	Хранение учётных записей пользователей и администраторов	При регистрации, входе, оформлении заказа
«Profiles»	ОИ	Хранение анкет пользователей с 15 параметрами	При заполнении/обновлении анкеты, при подборе
«Products»	НСИ	Хранение каталога косметических средств	При подборе, в каталоге, в корзине, в заказах
«Recommendations»	ОИ	История всех подборок пользователя	При каждом подборе, при просмотре ЛК
«Favorites»	ОИ	Избранные товары пользователя	При добавлении/удалении из избранного
«Carts»	ОИ	Текущая корзина пользователя	При добавлении/удалении товаров, при оформлении заказа
«PromoCodes»	НСИ	Промокоды для скидок	При оформлении заказа
«GiftCards»	ОИ	Подарочные карты пользователей	При покупке и применении карты
«OrderGiftCards»	ОИ	Связь заказов и карт	При оформлении заказа с картой
«Orders»	ОИ	Оформленные заказы	При оформлении заказа, при просмотре истории
«OrderItems»	ОИ	Товары в составе заказа	При оформлении и просмотре заказов

Администратор обновляет каталог через админ-панель – изменения автоматически отражаются в подборе.

2.1.7 Формы выходных документов

В системе предусмотрены несколько типов выходных форм, они адаптированы под мобильные устройства и работают в «PWA-режиме».

Справочные формы предназначены для просмотра данных. Каталог товаров, представленный на рисунке 4, содержит карточки с фото, названием, брендом, ценой и рейтингом; предусмотрена фильтрация по категориям, подкатегориям и цене, а также поиск по названию. Запросные формы формируются по запросу пользователя: страница подборки на рисунке 5 отображает ТОП-10 товаров с рейтингом совместимости и пояснениями, а также блок «Также может вам подойти».

Регламентированные формы формируются автоматически: личный кабинет на рисунке 6 содержит историю подборок с датами, раздел

[8] Составлено автором

«Избранное» и данные анкеты. Контрольные формы предназначены для администратора: админ-панель на рисунке 7 отображает статистику системы и таблицу товаров с возможностью редактирования и удаления.

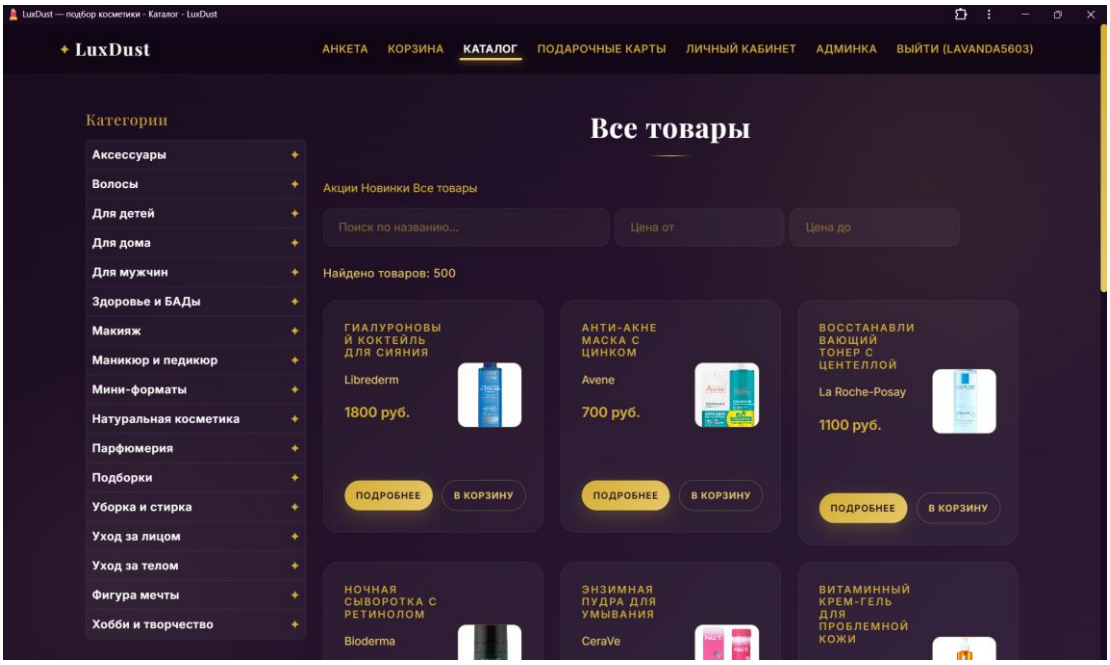


Рисунок 4 – Каталог товаров[9]

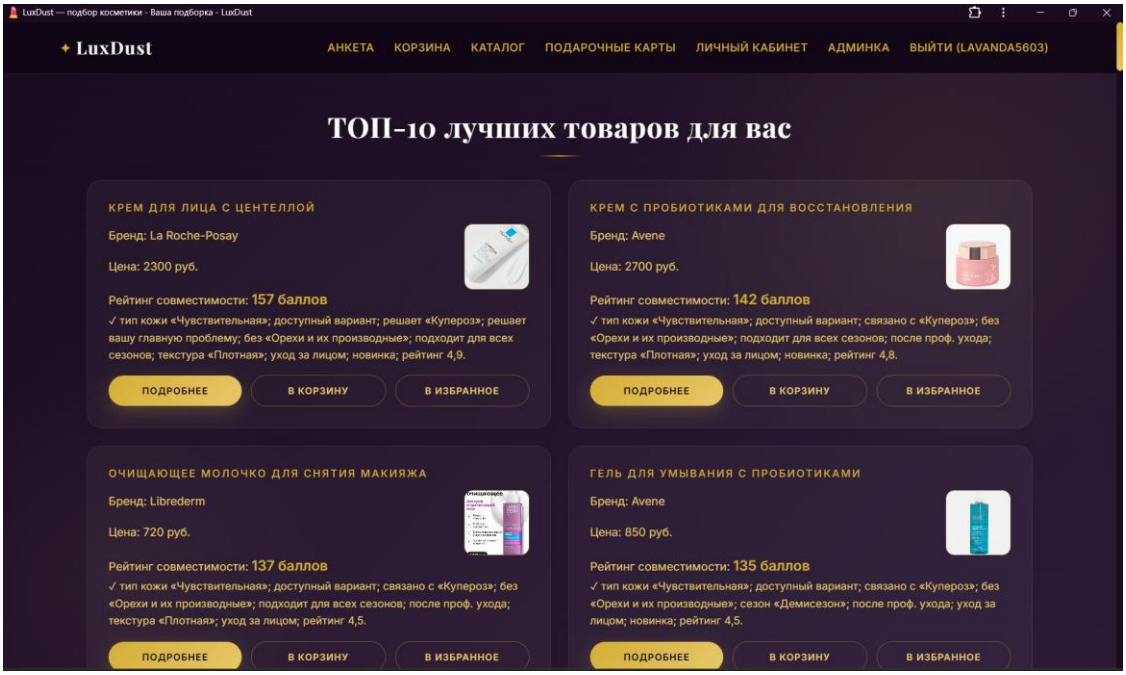


Рисунок 5 – Страница подборки[10]

[9] Составлено автором
[10] Составлено автором

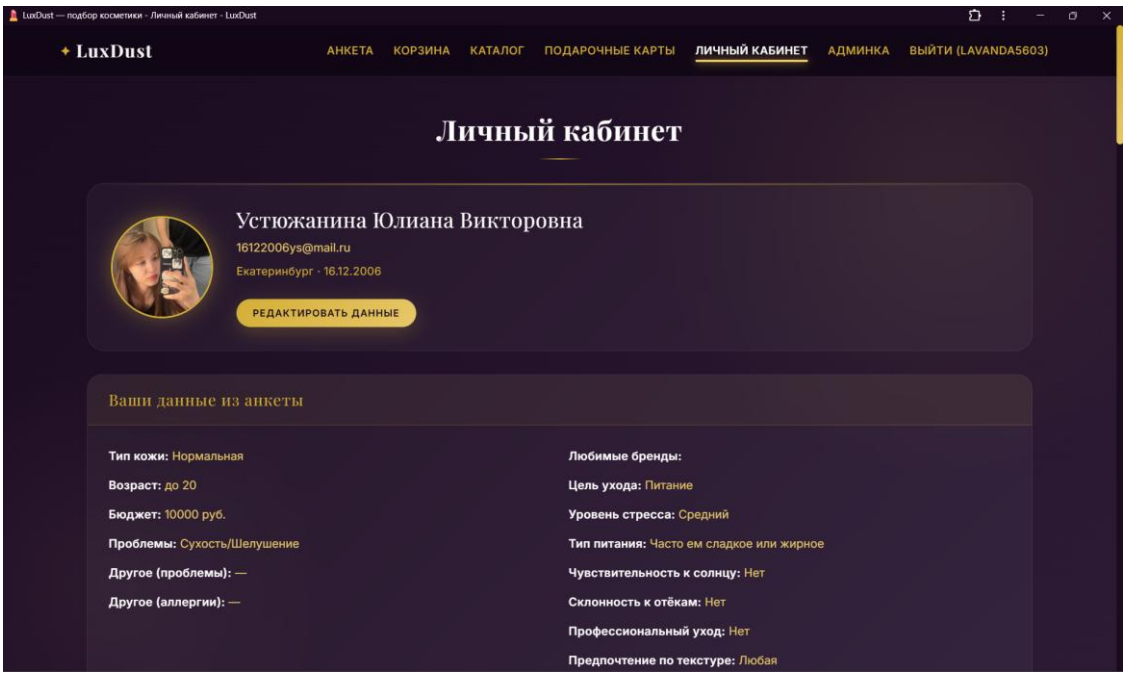


Рисунок 6 – Личный кабинет[11]

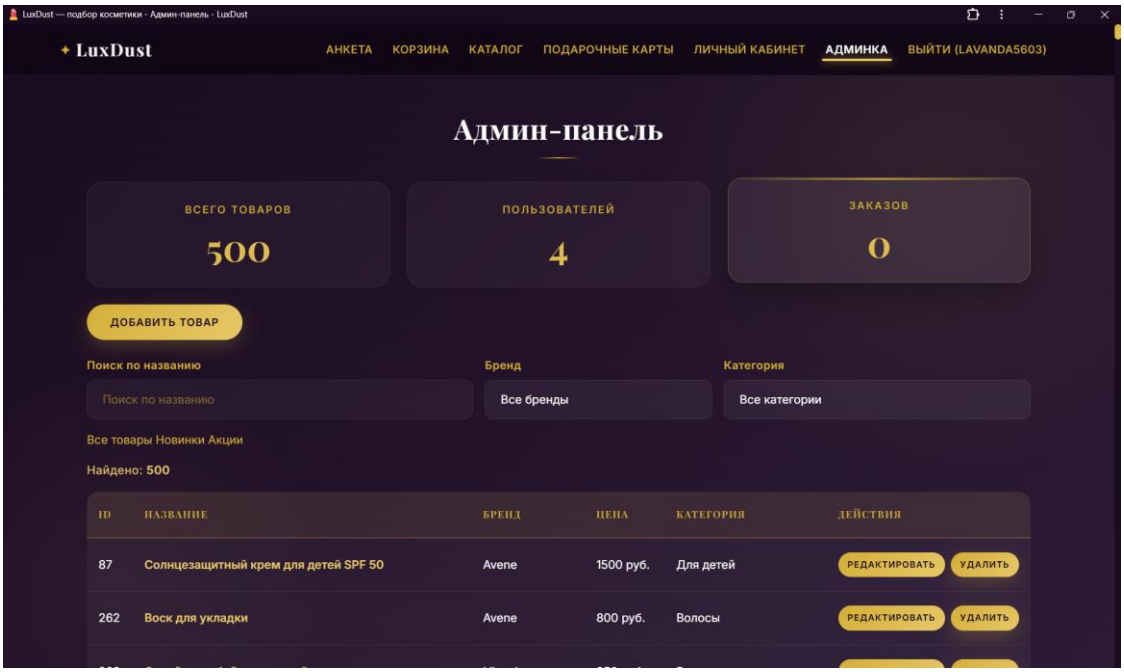


Рисунок 7 – Админ-панель[12]

2.2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.2.1 Структура программного обеспечения

Я разделила код на несколько слоёв, чтобы избежать дублирования и упростить поддержку. Слой «Controllers» содержит 7 контроллеров:

[11] Составлено автором
[12] Составлено автором

«HomeController», «QuizController», «AccountController», «CatalogController», «CheckoutController», «AdminController» и «GiftCardsController». В слое «Models» – 13 моделей данных и 3 «ViewModel».

Слой «Services» включает «RecommendationService» с логикой подбора (17 методов) и «RecommendationConstants» со словарями аллергенов, целей ухода и связей между проблемами. Слой «Data» представлен классом «ApplicationDbContext».

Слой «Views» содержит «Razor-страницы», сгруппированные по папкам. Слой «wwwroot» хранит статику: «CSS», «JavaScript», изображения, «manifest.json» и «service-worker.js».

Структура проекта показана на рисунке 8.

«ViewModel» вынесены отдельно: «RegisterViewModel» – для валидации регистрации, «AdminProductViewModel» – для списков брендов и категорий в админке.

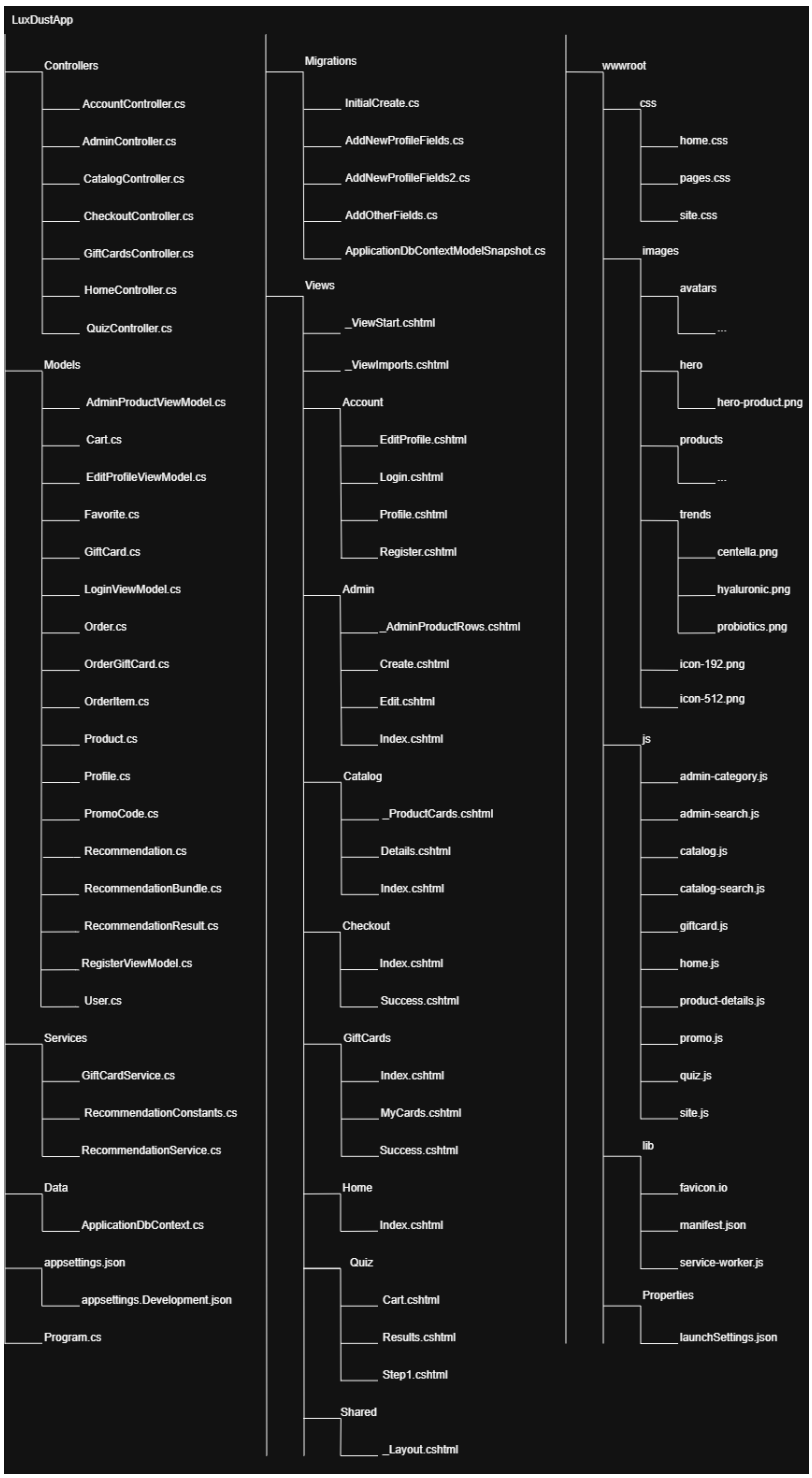


Рисунок 8 – Дерево программных модулей[13]

Для описания структуры программного обеспечения построена «UML-диаграмма» классов, продемонстрированная на рисунке 9. На диаграмме отражены четыре группы классов: модели данных, сервисы, контекст базы данных и контроллеры.

[13] Составлено автором

Модели данных («User», «Profile», «Product») описывают сущности предметной области. Класс «User» содержит информацию о пользователе. Класс «Profile» хранит 15 параметров анкеты. Класс «Product» описывает косметическое средство.

Сервис «RecommendationService» – центральный класс бизнес-логики. Он содержит 17 методов: главный «GetRecommendationsWithReasons()» и вспомогательные «CheckSkinType()», «CheckAllergies()», «CheckGoal()» и другие.

Контекст базы данных «ApplicationDbContext» наследуется от «DbContext» и содержит по одному свойству «DbSet<T>» для каждой модели [10].

Контроллер «QuizController» обрабатывает «HTTP-запросы», связанные с анкетой и подбором.

Связи между классами: «User» связан с «Profile» отношением 1:1, User и «Product» – с «Recommendation», «Favorite», «Cart» отношением 1:N. Контроллер зависит от сервиса и контекста, сервис – от контекста, контекст агрегирует модели.

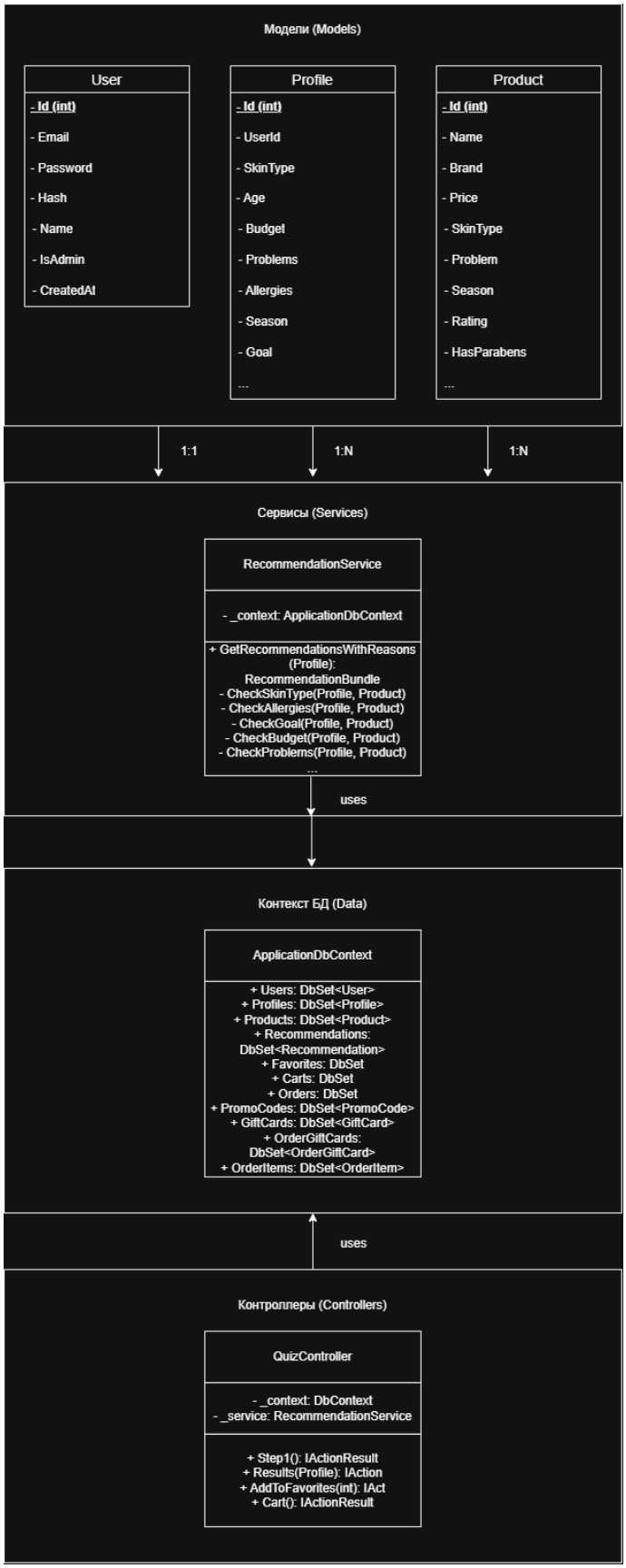


Рисунок 9 – «UML-диаграмма» классов[14]

[14] Составлено автором

2.2.2 Алгоритм подбора – ядро системы

Алгоритм подбора – это ядро всей системы. Он принимает 15 параметров из анкеты и для каждого из 500 товаров вычисляет рейтинг совместимости по формуле (1). Чем выше рейтинг, тем лучше товар подходит пользователю.

Формула рейтинга:

$$\text{Score} = \sum_{i=1}^{16} w_i * f_i(\text{profile}, \text{product}), \quad (1)$$

где «Score» – итоговый рейтинг совместимости; «w_i» – вес «i-го» фактора; «f_i» – функция, возвращающая 1 при совпадении, 0 при несовпадении или штраф при конфликте; «profile» – данные анкеты пользователя; «product» – характеристики товара.

Веса факторов приведены в таблице 6.

Таблица 6 – Весовые коэффициенты алгоритма подбора[15]

№	Фактор	Вес (баллы)	Логика
1	Тип кожи	+40	Точное совпадение
2	Связанные типы кожи	+15…+25	Обезвоженная – Сухая, Купероз – Чувствительная
3	Бюджет	+30 / -50	В бюджет – бонус, выше – штраф
4	Проблема	+30 (+10 при одной, +20 при двух, +30 при всех)	Совпадение или связанная проблема, бонус за количество
5	Аллергии	-50 / +10 (при 2+ -100)	Штраф за каждый аллерген, критичный штраф при 2+
6	Сезон	+15 / -10	Совпадение или конфликт
7	Любимые бренды	+25	Совпадение бренда
8	Цель ухода	+20	Совпадение цели
9	Возраст	+10…+30	5 групп с разными баллами
10	Уровень стресса	+5…+15	Высокий стресс – успокаивающие средства
11	Тип питания	+12…+15	Вегетарианство – веган-косметика
12	Чувствительность к солнцу	+20	«SPF-защита»
13	Склонность к отёкам	+10…+20	Средства от отёков
14	Профессиональный уход	-10 / +10	Штраф за скрабы
15	Текстура	+10	Совпадение текстуры
16	Многоступенчатый уход	+10…+15 / -50	Бонус за сыворотки, штраф при неготовности
-	Бонусы	+5	Новинка, акция, высокий рейтинг

[15] Составлено автором

Блок-схема алгоритма приведена на рисунке 10.

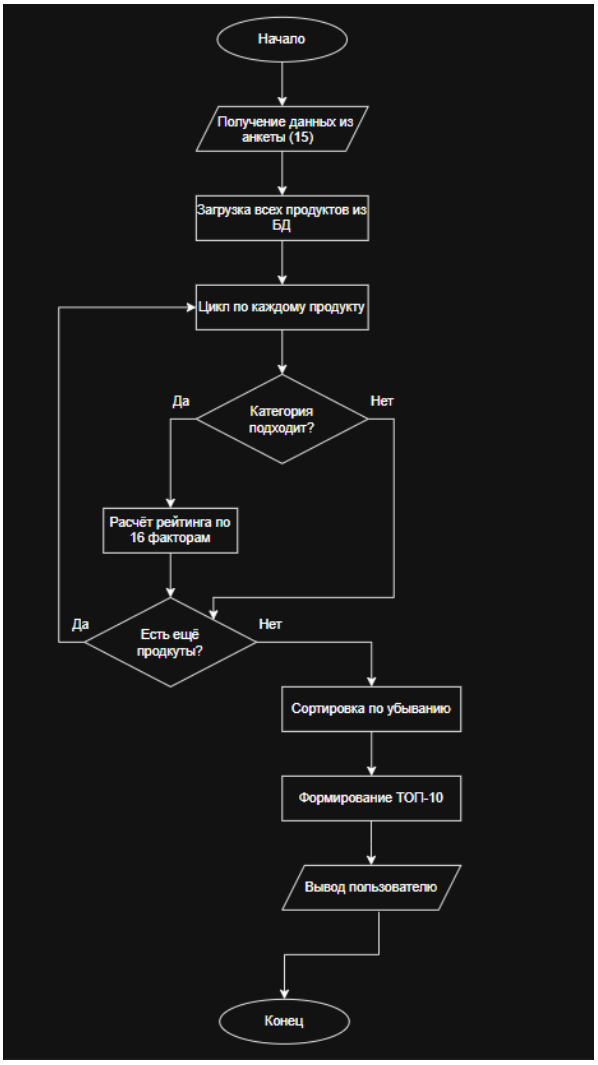


Рисунок 10 – Блок-схема алгоритма подбора[16]

Алгоритм работает следующим образом. Сначала получаются данные из анкеты, затем загружаются все 500 товаров из базы. Запускается цикл по каждому товару: проверяется категория (подходит ли товар для ухода за лицом), и, если да – вычисляется рейтинг по 16 факторам. После проверки всех товаров выполняется сортировка по убыванию рейтинга, формируется ТОП-10 и блок «Также может вам подойти». Результат выводится пользователю.

Пример расчёта рейтинга. Пользователь указал: тип кожи – обезвоженная, возраст – 30-40, бюджет – 1000-3000 руб., проблема – купероз, аллергия – силиконы, сезон – демисезон, цель – восстановление, текстура –

[16] Составлено автором

плотная. Рассматривается товар «Сыворотка с центеллой» («Avene», 2300 руб., тип кожи «Чувствительная», проблема «Купероз», без силиконов, сезон «Круглый год», цель «Восстановление»). Расчёт:

– тип кожи: обезвоженная – чувствительная: +25;

– бюджет: 2300 руб. попадает в диапазон: +30;

– проблема: «Купероз» совпадает: +30;

– аллергия: без силиконов: +10;

– сезон: «Круглый год»: +12;

– цель: «Восстановление»: +20;

– текстура: «Плотная»: +10;

– бонусы: новинка (+5), рейтинг 4.9 (+5).

Итого «Score» = 147 баллов. Товар попадёт в ТОП-10.

2.2.3 Логика обработки ответов пользователя

Каждый ответ в анкете обрабатывается отдельным методом в «RecommendationService». Всего таких методов 17: «CheckSkinType()», «CheckAge()», «CheckBudget()», «CheckProblems()», «CheckAllergies()», «CheckSeason()», «CheckBrands()», «CheckGoal()», «CheckStress()», «CheckDiet()», «CheckSunSensitivity()», «CheckEdema()», «CheckProfessionalCare()», «CheckTexture()», «CheckMultiStep()», «CheckCategory()», «CheckBonuses()».

Я разбила логику на методы, чтобы избежать дублирования и упростить поддержку. Пример обработки аллергий показан в приложении В.

Подход через словарь позволил убрать длинную цепочку «if-else». Аналогично реализованы цели ухода и связанные проблемы. Дополнительно добавлена проверка категорий: при запросе ухода за лицом товары из категорий «Уход за телом», «Волосы», «Макияж» и других исключаются из подбора.

2.2.4 Моделирование пользовательских сценариев

Основной пользовательский сценарий смоделирован в нотации «BPMN» и представлен на рисунке 11. Диаграмма разделена на две дорожки: «Пользователь» и «Система».

Сценарий выглядит так. Пользователь открывает сайт, заполняет анкету из 15 шагов и нажимает «Получить подборку». Система обрабатывает данные анкеты, считает рейтинг каждого товара по 16 факторам, формирует ТОП-10 и блок «Также может вам подойти». Далее возможны два пути: добавить товары в корзину, применить промокод или подарочную карту и оформить заказ либо сохранить подборку в личном кабинете.

Все задачи оформлены стандартными элементами «BPMN»: задачи («Task») – скруглёнными прямоугольниками, развилка («Gateway») – ромбом, начало и конец («Start Event», «End Event») – кругами.

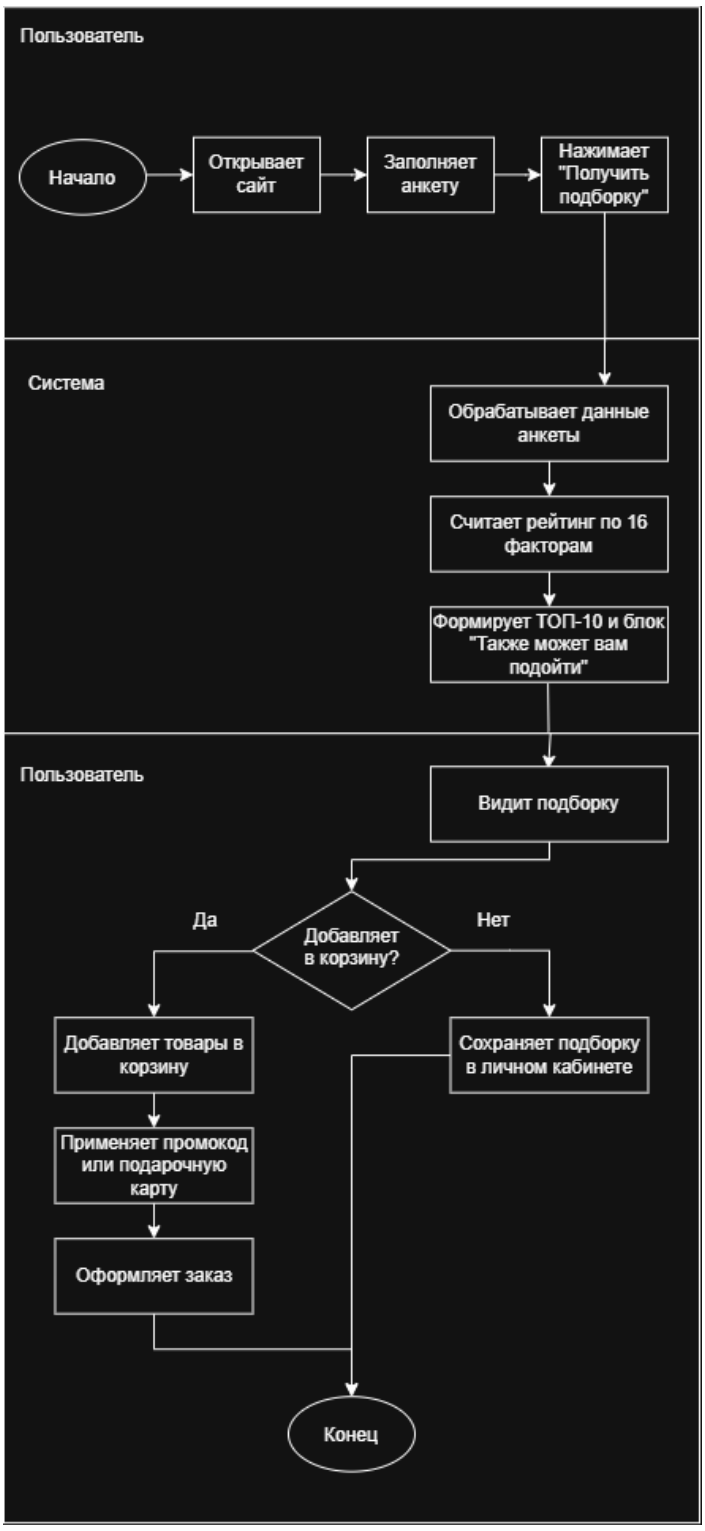


Рисунок 11 – «BPMN-диаграмма» пользовательского сценария[17]

[17] Составлено автором

2.3 УПРАВЛЕНИЕ ПРОЕКТОМ

2.3.1 Календарный план-график работ

Работа распланирована на 10 недель – с 2 сентября по 11 ноября 2026 года. Каждая неделя посвящена отдельному этапу:

– 1 неделя (2-8 сентября): настройка среды, установка «PostgreSQL», создание проекта, подключение «GitHub»;

– 2 неделя (9-15 сентября): создание моделей данных, контекста БД и первой миграции;

– 3 неделя (16-22 сентября): разработка главной страницы, навигации и базовых стилей;

– 4 неделя (23-29 сентября): разработка анкеты из 15 шагов и «JavaScript-логики»;

– 5 неделя (30 сентября - 6 октября): разработка алгоритма подбора, первые тесты;

– 6 неделя (7-13 октября): личный кабинет, сохранение подборок, избранное;

– 7 неделя (14-20 октября): регистрация, авторизация, корзина;

– 8 неделя (21-27 октября): «PWA», каталог с подкатегориями, поиск, пагинация;

– 9 неделя (28 октября - 3 ноября): админ-панель, оформление заказов;

– 10 неделя (4-11 ноября): тестирование, отладка и оформление отчёта.

Календарный план-график приведён на рисунке 12.

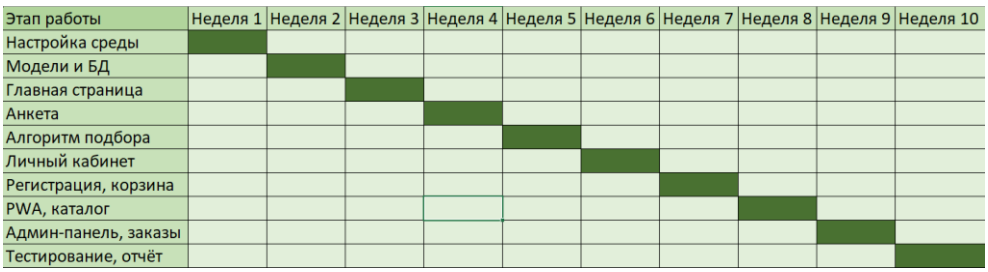


Рисунок 12 – Диаграмма Ганта[18]

[18] Составлено автором

2.3.2 Бюджет на разработку

Проект реализован без финансовых затрат на лицензии. Все инструменты – бесплатные: «Visual Studio Community», «PostgreSQL», «GitHub», «Bootstrap», «Font Awesome», «draw.io», «dbdiagram.io», «pgAdmin».

Единственные затраты – время. При работе 1,5-2 часа в день общие временные затраты составили около 60-80 часов за 2 месяца.

2.3.3 Оценка рисков и способы их минимизации

В процессе разработки я столкнулась со следующими рисками, они приведены в таблице 7.

Таблица 7 – Риски проекта и способы их минимизации[19]

Риск	Что произошло	Как решено
Ошибка с паролем «PostgreSQL»	При «Update-Database» возникла ошибка «28P01: password authentication failed»	Найден «pg_hba.conf», метод изменён на trust, пароль сброшен через «ALTER USER», метод возвращён на «scram-sha-256»
Ошибка с типом «DateTime»	«PostgreSQL» требовал «UTC» для «timestamp with time zone»	Замена «DateTime.Now» на «DateTime.UtcNow»
Ошибка с «null-значениями»	При чтении «NULL» из БД возникало исключение	Все текстовые поля в «Profile» сделаны «nullable»
Неправильная работа алгоритма	Товары из «Ухода за телом» попадали в подборку для лица	Исключены лишние категории в «RecommendationService»
Проблемы с адаптивностью	На мобильных устройствах вёрстка ломалась	Использована «Bootstrap-сетка» и медиа-запросы
Дублирование логики	Методы «CheckGoal» и «CheckAllergies» были на 100+ строк	Переписаны через словари «goalRules» и «allergenMap»
«PWA» не устанавливался	Отсутствовали «manifest.json» и «service-worker.js»	Добавлены оба файла и иконки
Предупреждения компилятора	«Visual Studio» выдавал 57 предупреждений («CS8618, CS8602, CS8625, CS8604, CS8619»)	Добавлены = «null!»; для навигационных свойств, ?. и ?? «» для разыменования, «string?» для необязательных параметров

Все эти риски возникли в процессе разработки. Ошибка с паролем «PostgreSQL» заняла около часа. Ошибка с «DateTime.Now» показала, что все даты нужно хранить в «UTC». Ошибка с «nullable-полями» заставила пересмотреть модель «Profile».

[19] Составлено автором

3 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 ПРОГРАММНО-ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

3.1.1 Структура программного обеспечения

Разработанная информационная система реализована как веб-приложение на «ASP.NET Core MVC» с использованием «Entity Framework Core» и базы данных «PostgreSQL». Приложение построено по паттерну «Model-View-Controller» и разделено на шесть слоёв: контроллеры, модели, сервисы, контекст базы данных, представления и статические файлы. Такое разделение обеспечивает чёткое разграничение данных, бизнес-логики и отображения.

3.1.2 Спецификации программных модулей

Ключевые программные модули системы можно разделить на три блока.

Первый – модуль доступа к базе данных. Класс «ApplicationDbContext» наследуется от «DbContext» («Entity Framework Core») и содержит по одному свойству «DbSet<T>» для каждой модели. Через него приложение взаимодействует с «PostgreSQL». Фрагмент класса «ApplicationDbContext» приведён в Приложении Г.

Второй – модуль подбора косметических средств. Центральный модуль системы, класс «RecommendationService», содержит 17 методов: главный «GetRecommendationsWithReasons()» и 16 вспомогательных методов «Check*()». Главный метод принимает объект «Profile» и возвращает «RecommendationBundle» – результат подбора. Фрагмент метода «GetRecommendationsWithReasons()» представлен в Приложении Д. Алгоритм работает так: загружает все товары из базы, отсекает неподходящие категории (если пользователь запросил уход за лицом), для каждого оставшегося товара вызывает 17 методов проверки, суммирует баллы в «Score», сортирует по убыванию, формирует ТОП-10 и блок «Также может вам подойти».

Третий – модуль обработки анкеты и подбора. Контроллер «QuizController» принимает данные анкеты, вызывает сервис подбора, сохраняет историю подборок и обновляет профиль пользователя. Фрагмент метода «Results()» контроллера «QuizController» показан в Приложении Е.

Метод проверяет авторизацию пользователя, вызывает сервис подбора, сохраняет 10 лучших товаров в историю подборок, обновляет анкету (если она уже была) и возвращает представление с результатом.

Остальные модули системы:

– «HomeController» – отображение главной страницы;

– «QuizController» – анкета, подбор, корзина, избранное (уже упомянут выше);

– «AccountController» – регистрация, вход, выход, личный кабинет;

– «CatalogController» – каталог, поиск, фильтрация, пагинация;

– «CheckoutController» – оформление заказа и страница подтверждения;

– «GiftCardsController» – покупка и просмотр подарочных карт;

– «AdminController» – «CRUD»-операции с товарами.

3.1.3 Пользовательский интерфейс

Интерфейс системы построен на «HTML», «CSS» («Bootstrap 5.3») и «JavaScript». Все страницы адаптированы под мобильные устройства и работают в режиме «PWA». Основные экраны системы: главная страница, анкета, страница подборки, корзина, оформление заказа, личный кабинет и админ-панель [11].

Главная страница содержит приветствие, краткое описание системы, кнопку «Начать подбор» и навигационное меню со ссылками на каталог, корзину, личный кабинет, вход и регистрацию.

Анкета – центральный элемент системы. Она состоит из 15 шагов, они появляются по одному: после ответа на текущий шаг открывается следующий. Сверху расположен прогресс-бар, показывающий на каком шаге находится пользователь. Кнопка «Продолжить» переключает шаги, а на последнем

меняется на «Получить подборку». Вся «JavaScript-логика» вынесена в отдельный файл «quiz.js». Внешний вид анкеты приведён на рисунке 13.

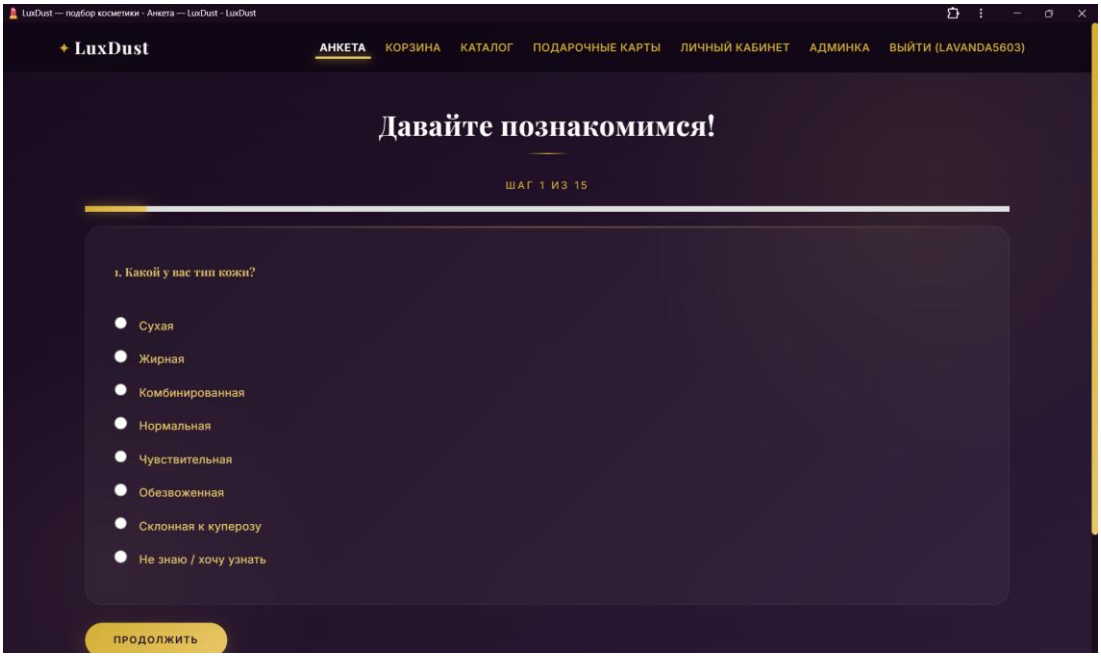


Рисунок 13 – Страница анкеты (первый шаг)[20]

Страница подборки отображает результат работы алгоритма. Пользователь видит ТОП-10 лучших товаров с фотографиями, названием, брендом, ценой и рейтингом совместимости. Под каждым товаром указаны причины, по которым он подошёл и предупреждения. Ниже расположен блок «Также может вам подойти» – товары, занявшие остальные места.

Корзина хранит выбранные товары. Добавление товаров в корзину реализовано через «AJAX» – без перезагрузки страницы. В корзине отображается список товаров с ценой и количеством, итоговая стоимость, а также кнопка «Оформить заказ». Удаление товаров выполняется с подтверждением. Внешний вид корзины приведён на рисунке 14.

[20] Составлено автором

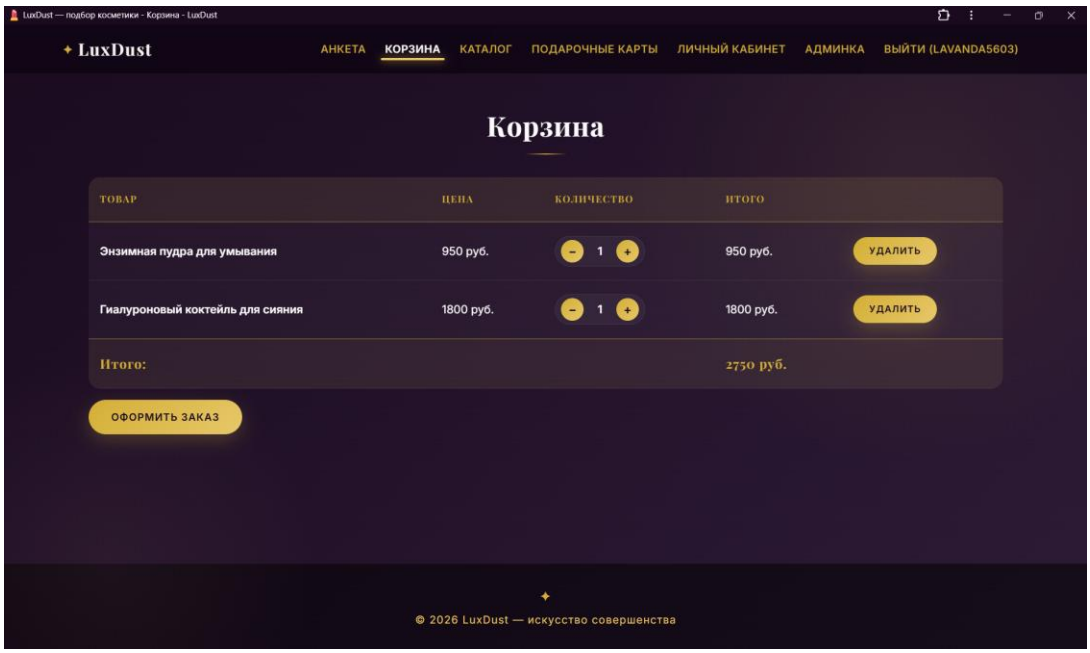


Рисунок 14 – Корзина с товарами[21]

Оформление заказа представлено страницей с формой, где пользователь выбирает способ доставки, дату доставки, время доставки, а также указывает телефон и комментарий. При выборе самовывоза поле «Адрес» скрывается с помощью «JavaScript». Внешний вид страницы оформления заказа приведён на рисунке 15.

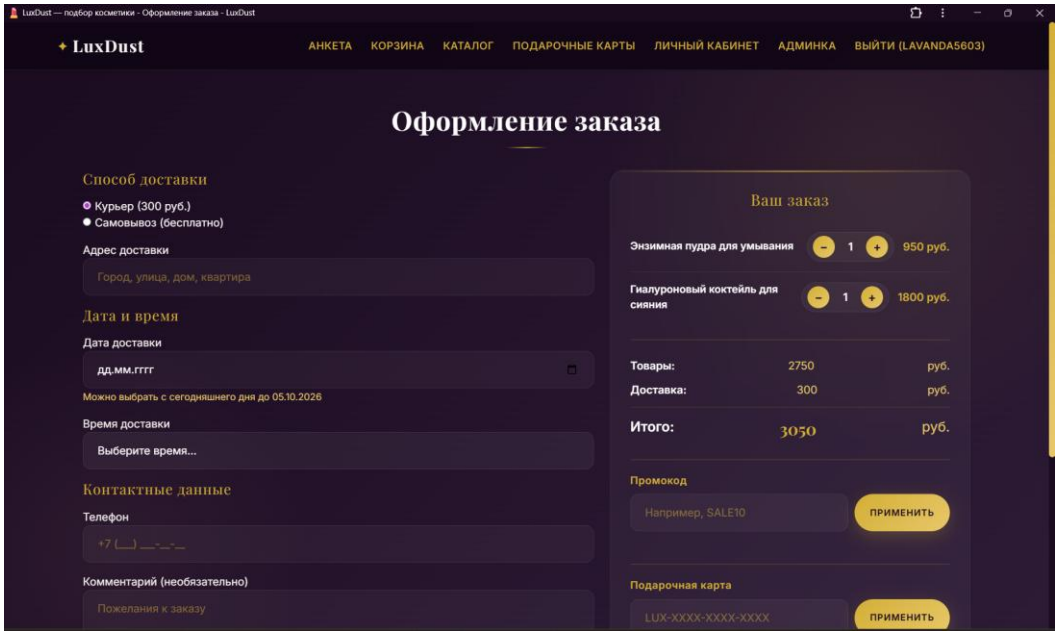


Рисунок 15 – Страница оформления заказа[22]

[21] Составлено автором
[22] Составлено автором

Страница «Спасибо за заказ» отображается после оформления. Она содержит номер заказа, способ доставки, дату и время, итоговую сумму и текущий статус. Статус заказа обновляется автоматически в зависимости от даты доставки и способа получения. Внешний вид страницы приведён на рисунке 16.

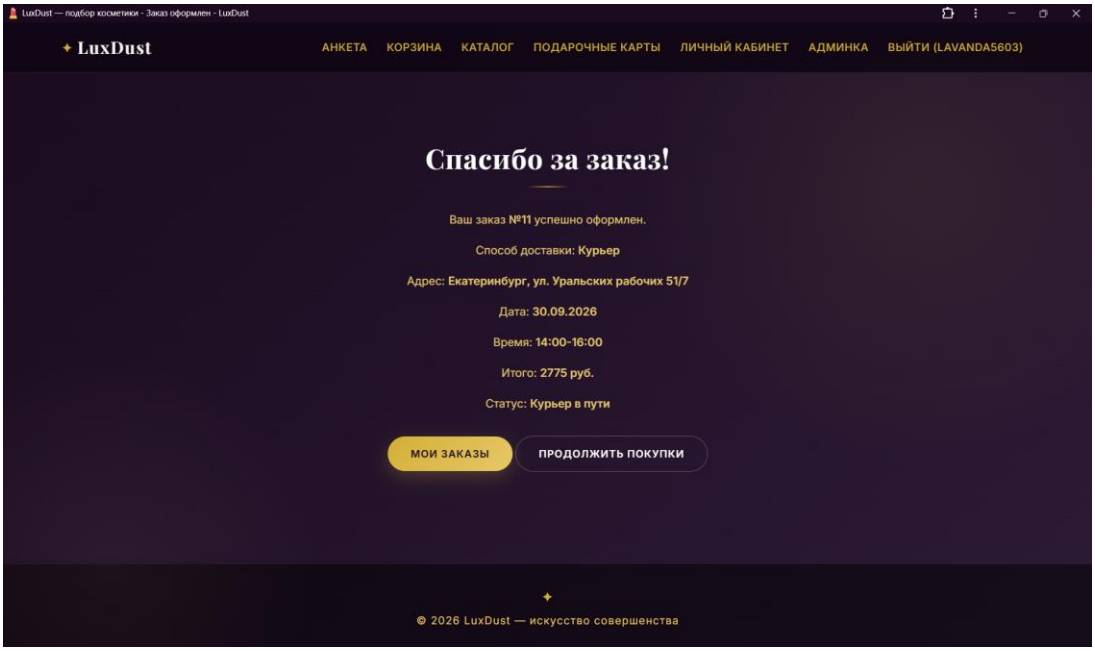


Рисунок 16 – Страница «Спасибо за заказ»[23]

Личный кабинет содержит данные анкеты пользователя с возможностью редактирования, историю подборок (с датами и составом), раздел «Избранное» и историю заказов.

Админ-панель доступна только администратору. Она содержит статистику системы (всего товаров, пользователей, заказов) и таблицу всех товаров с возможностью редактирования и удаления.

Диаграмма последовательности «UML». Взаимодействие компонентов системы при подборе показано на диаграмме последовательности, изображённой на рисунке 17. Пользователь заполняет анкету, браузер отправляет «POST-запрос» на сервер, «QuizController» вызывает «RecommendationService», тот обращается к «ApplicationDbContext» и «PostgreSQL», получает список товаров, рассчитывает «Score» по 16 факторам,

[23] Составлено автором

формирует подборку и возвращает её в контроллер, который сохраняет историю и отображает результат пользователю.

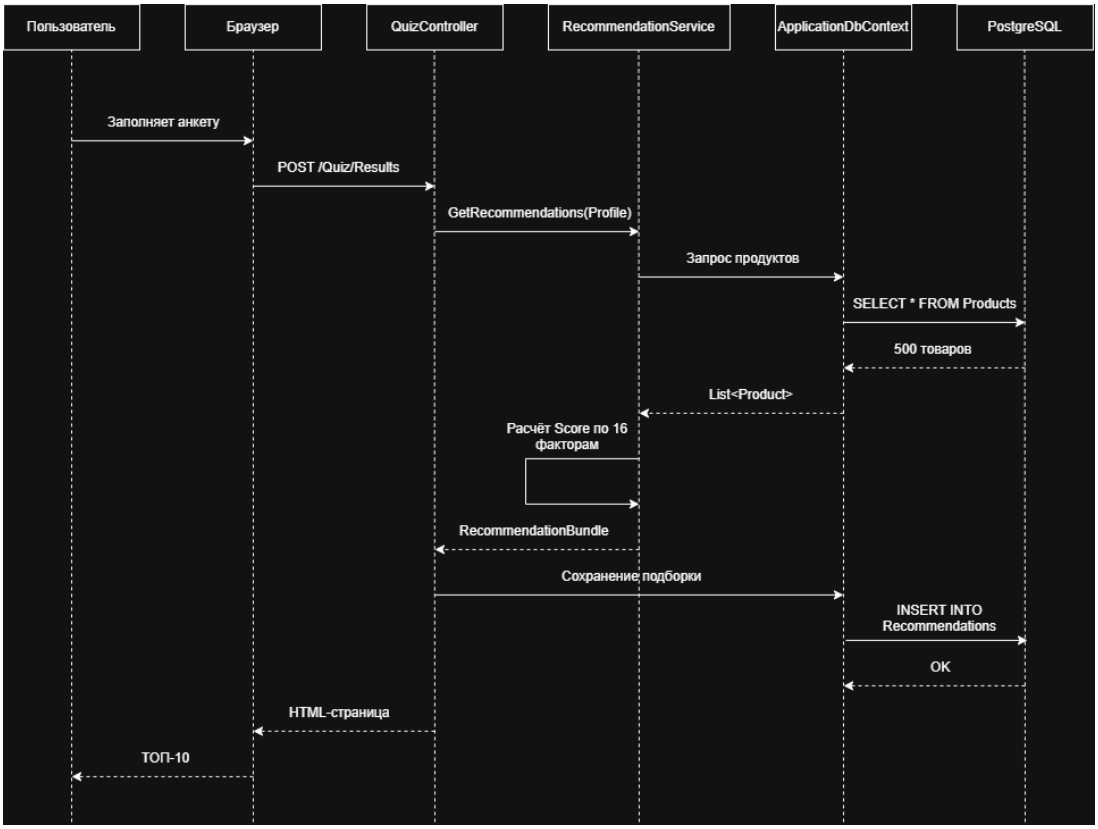


Рисунок 17 – Диаграмма последовательности «UML»[24]

3.1.4 Обеспечение информационной безопасности

Информационная безопасность системы обеспечивается на нескольких уровнях: аутентификация, хеширование паролей, разграничение прав доступа, защита персональных данных и валидация входных данных.

Используется «Cookie-аутентификация» «ASP.NET Core». После успешного входа пользователя сервер создаёт зашифрованный «Cookie», который хранится в браузере. При каждом запросе «Cookie» автоматически отправляется на сервер, и система определяет, кто именно обратился. Если пользователь не авторизован, доступ к защищённым страницам (анкета, личный кабинет, корзина) блокируется – происходит перенаправление на страницу входа.

[24] Составлено автором

Пароли пользователей не хранятся в открытом виде. Для хеширования используется библиотека «BCrypt.Net-Next». При регистрации пароль преобразуется в хеш, который сохраняется в базе данных. При входе введённый пароль хешируется повторно и сравнивается с сохранённым. Это означает, что даже администратор базы данных не может узнать реальные пароли пользователей [6].

Предусмотрены три роли: гость, пользователь и администратор. Каждая роль имеет свой набор доступных функций. Распределение прав доступа приведено в таблице 8.

Таблица 8 – Распределение прав доступа[25]

Функция	Гость	Пользователь	Администратор
Просмотр каталога	Да	Да	Да
Просмотр карточки товара	Да	Да	Да
Прохождение анкеты	Нет	Да	Да
Получение подборки	Нет	Да	Да
Добавление в корзину	Нет	Да	Да
Добавление в избранное	Нет	Да	Да
Оформление заказа	Нет	Да	Да
Просмотр личного кабинета	Нет	Да	Да
Просмотр истории подборок	Нет	Да	Да
Редактирование анкеты	Нет	Да	Да
Админ-панель	Нет	Нет	Да
Добавление товара	Нет	Нет	Да
Редактирование товара	Нет	Нет	Да
Удаление товара	Нет	Нет	Да
Просмотр статистики	Нет	Нет	Да

Система хранит имя, «email» и хеш пароля пользователя. Согласно Федеральному закону №152-ФЗ «О персональных данных», эти данные защищены: передаются по «HTTPS», хранятся в защищённой базе данных, не передаются третьим лицам. При регистрации пользователь даёт согласие на обработку персональных данных.

При регистрации система проверяет «email» на уникальность, пароль на сложность (6+ символов, буквы, цифры, спецсимволы), совпадение пароля и заполненность полей. При работе с анкетой и админ-панелью также проверяется корректность вводимых данных, что предотвращает ошибки и

[25] Составлено автором

защищает от «SQL-инъекций» (используется «Entity Framework Core» с параметризованными запросами).

В продакшене работает по протоколу «HTTPS», который шифрует трафик между браузером и сервером. Это защищает данные от перехвата.

3.2 ТЕСТИРОВАНИЕ, ОЦЕНКА КАЧЕСТВА И ВНЕДРЕНИЕ

3.2.1 Исходные данные и результаты работы системы

Для проверки работоспособности системы было проведено тестирование на пяти сценариях, охватывающих разные типы кожи, возрастные группы и проблемы.

В первом сценарии пользователь указал тип кожи «Сухая», возраст 30-40, бюджет 1000-3000 руб., проблему «Сухость/Шелушение», аллергию «Отдушки», сезон «Зима» и цель «Увлажнение». Система выдала в ТОП-3 «Гиалуроновый коктейль для сияния» («Librederm», 1800 руб.), «Крем-суфле для сухой кожи» («La Roche-Posay», 2400 руб.) и «Увлажняющий тоник с гиалуроновой кислотой» («La Roche-Posay», 1100 руб.). Все средства увлажняющие, без отдушек, для сухой кожи.

Во втором сценарии пользователь указал тип кожи «Жирная», возраст до 20, бюджет до 1000 руб., проблему «Акне», аллергию «Спирт», сезон «Лето» и цель «Очищение». Система выдала «Гель для умывания с цинком» («CeraVe», 900 руб.), «Гель для умывания с салициловой кислотой» («CeraVe», 950 руб.) и «Гель для умывания с чайным деревом» («Avene», 850 руб.). Средства очищающие, без спирта, для жирной кожи.

В третьем сценарии пользователь указал тип кожи «Нормальная», возраст 50+, бюджет 3000-10000 руб., проблему «Морщины», отсутствие аллергий, сезон «Круглый год» и цель «Антивозрастной». Система выдала «Антивозрастной крем с коллагеном» («Bioderma», 3200 руб.), «Сыворотка с коллагеном» («Librederm», 2700 руб.) и «Сыворотка с пептидами для упругости» («Librederm», 2900 руб.). Все средства антивозрастные, для зрелой кожи.

В четвёртом сценарии пользователь указал тип кожи «Чувствительная», возраст 20-30, бюджет 1000-3000 руб., проблему «Купероз», аллергию «Эфирные масла», сезон «Демисезон», цель «Восстановление» и тип питания «Вегетарианство». Система выдала «Сыворотка с центеллой для восстановления» («Avene», 2300 руб.), «Сыворотка с пробиотиками» («CeraVe», 2500 руб.) и «Крем для лица с центеллой» («La Roche-Posay», 2300 руб.). Все средства успокаивающие, без эфирных масел, подходят для чувствительной кожи.

В пятом сценарии пользователь указал тип кожи «Комбинированная», возраст 30-40, бюджет 3000-10000 руб., проблемы «Акне, Пигментация, Морщины, Тусклый цвет», аллергии «Парабены, Отдушки, Спирт, Силиконы», сезон «Круглый год», цель «Восстановление, Сияние» и уровень стресса «Высокий». Система выдала «Сыворотка с растительными стволовыми клетками» («Bioderma», 3500 руб.), «Сыворотка с цинком для проблемной кожи» («CeraVe», 2100 руб.) и «Ночная сыворотка с ретинолом» («Bioderma», 3200 руб.). Все средства без парабенов, отдушек, спирта и силиконов, подходят для комбинированной кожи и решают несколько проблем.

Сводные результаты тестирования продемонстрированы в таблице 9.

Таблица 9 – Результаты тестирования[26]

№	Профиль	ТОП-3 товара	Ошибки
1	Сухая кожа, 30-40, 1000-3000 руб., аллергия на отдушки	Гиалуроновый коктейль для сияния («Librederm», 1800 руб.); Крем-суфле для сухой кожи («La Roche-Posay», 2400 руб.); Увлажняющий тоник с гиалуроновой кислотой («La Roche-Posay», 1100 руб.)	нет
2	Жирная кожа, до 20, до 1000 руб., аллергия на спирт	Гель для умывания с цинком («CeraVe», 900 руб.); Гель для умывания с салициловой кислотой («CeraVe», 950 руб.); Гель для умывания с чайным деревом («Avene», 850 руб.)	нет
3	Нормальная кожа, 50+, 3000-10000 руб., антивозрастной уход	Антивозрастной крем с коллагеном («Bioderma», 3200 руб.); Сыворотка с коллагеном («Librederm», 2700 руб.); Сыворотка с пептидами для упругости («Librederm», 2900 руб.)	нет

[26] Составлено автором

Продолжение таблицы 9

№	Профиль	ТОП-3 товара	Ошибки
4	Чувствительная кожа, 20-30, 1000-3000 руб., купероз, вегетарианство	Сыворотка с центеллой для восстановления («Avene», 2300 руб.); Сыворотка с пробиотиками («CeraVe», 2500 руб.); Крем для лица с центеллой («La Roche-Posay», 2300 руб.)	нет
5	Комбинированная кожа, 30-40, 3000-10000 руб., много проблем и аллергий	Сыворотка с растительными стволовыми клетками («Bioderma», 3500 руб.); Сыворотка с цинком для проблемной кожи («CeraVe», 2100 руб.); Ночная сыворотка с ретинолом («Bioderma», 3200 руб.)	нет

3.2.2 Поэтапное описание работы с системой

Работа пользователя с системой выглядит следующим образом. Сначала он открывает главную страницу, нажимает «Регистрация», заполняет форму (имя, «email», пароль, подтверждение пароля) – система проверяет «email» на уникальность, пароль на сложность и сохраняет аккаунт. После входа пользователь нажимает «Начать подбор», открывается первый шаг анкеты, он отвечает на 15 вопросов, переключаясь кнопкой «Продолжить»; на последнем шаге кнопка меняется на «Получить подборку».

Затем система обрабатывает данные анкеты, рассчитывает рейтинг каждого товара по 16 факторам, формирует ТОП-10 и блок «Также может вам подойти», а пользователь видит подборку с рейтингом совместимости и пояснениями. После этого он добавляет понравившиеся товары в корзину («AJAX») или в избранное – кнопка меняется на «Добавлено ✓».

Перейдя в корзину, пользователь нажимает «Оформить заказ», выбирает способ доставки, дату и время, указывает телефон и подтверждает заказ. В завершение он может вернуться в личный кабинет, посмотреть историю подборок, избранное и историю заказов, а также отредактировать анкету.

Система поддерживает установку на телефон как «PWA». На рисунке 18 показана иконка приложения на домашнем экране телефона. На рисунке 19 показан полноэкранный режим работы приложения – без браузерной строки, как полноценное мобильное приложение [11].



Рисунок 18 – «PWA» на домашнем экране телефона[27]



Рисунок 19 – «PWA» в полноэкранном режиме[28]

3.2.3 Порядок внедрения проекта

Для внедрения системы я определила следующий порядок. Сначала нужно подготовить сервер: подойдёт «Windows» или «Linux», требуется «.NET 8.0 Runtime», «PostgreSQL 15+», не менее 2 ГБ ОЗУ и 10 ГБ на диске. Затем установить «PostgreSQL»: скачать с официального сайта, создать базу данных «LuxDustDB» и задать пароль пользователя «postgres». После этого развернуть приложение: скопировать файлы проекта на сервер, настроить строку

[27] Составлено автором
[28] Составлено автором

подключения в «appsettings.json» и выполнить «dotnet ef database update» для создания таблиц.

Далее настроить «HTTPS»: получить «SSL-сертификат» («Let's Encrypt») и настроить «Kestrel» или «IIS» для работы по защищённому протоколу. В завершение обучить администратора: показать, как добавлять товары, редактировать цены и смотреть статистику.

3.3 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА

3.3.1 Анализ затрат на ресурсное обеспечение

Все инструменты, использованные в проекте, – бесплатные: «Visual Studio Community», «PostgreSQL», «GitHub», «Bootstrap», «Font Awesome», «draw.io», «pgAdmin». Это существенное преимущество по сравнению с коммерческими аналогами: «MS SQL Server Standard» и «Visual Studio Professional».

Единственные затраты – время разработчика. При работе 1,5-2 часа в день общие временные затраты составили около 60-80 часов за 2 месяца.

Экономия бюджета пользователя рассчитывается по формуле (2):

$$\text{Экономия} = \text{Бюджет}_{\text{было}} - \text{Бюджет}_{\text{стало}} \quad (2)$$

где $\text{Бюджет}_{\text{было}}$ – сумма, которую пользователь тратил на косметику до использования системы; $\text{Бюджет}_{\text{стало}}$ – сумма, которую он тратит после.

3.3.2 Качественные и количественные эффекты

Для пользователя система даёт три эффекта. Во-первых, экономию времени: до использования системы человек тратил 3-4 часа на изучение составов и отзывов, а с системой – 5 минут на заполнение анкеты, то есть сокращение времени подбора в 36-48 раз. Во-вторых, экономию бюджета: при бюджете 5000 руб/мес пользователь экономит до 1500 руб/мес (30%), потому что система рекомендует только подходящие средства и не приходится тратить деньги на неподходящие. В-третьих, повышение точности подбора: алгоритм учитывает 16 факторов, что снижает вероятность ошибки по сравнению с выбором наугад или по совету консультанта.

Для магазина эффект тоже тройной. Во-первых, рост продаж: персонализированные рекомендации повышают доверие и увеличивают конверсию, прогнозируемый рост – 15-20%. Во-вторых, снижение возвратов: клиенты реже возвращают товары, потому что система подбирает именно то, что подходит, снижение возвратов – 25-30%. В-третьих, повышение лояльности: пользователь, который получил точную рекомендацию, возвращается в магазин снова, что создаёт долгосрочный эффект – повторные продажи.

Эргономический эффект – интерфейс интуитивно понятен, не требует специальной подготовки. Пользователь заполняет анкету за 5 минут, не читая инструкций. Все страницы адаптированы под мобильные устройства и работают в режиме «PWA».

Социальный эффект – система доступна с любого устройства (ПК, планшет, смартфон), работает в режиме «PWA», что делает её удобной для людей с разным уровнем технической подготовки.

3.3.3 Расчёт экономической эффективности

Все показатели экономической эффективности системы приведены в таблице 10.

Таблица 10 – Расчёт экономической эффективности[29]

Показатель	Значение
Затраты времени на разработку	60-80 часов
Затраты на инструменты	0 руб.
Экономия времени пользователя	с 3-4 часов до 5 минут
Экономия бюджета пользователя	до 30% (1500 руб/мес при бюджете 5000 руб)
Рост продаж магазина	+15-20%
Снижение возвратов	-25-30%
Срок окупаемости	не применимо (нет затрат)

Вывод. Система экономически эффективна, не требует затрат на инструменты, сокращает время подбора в 36-48 раз, экономит бюджет до 30%, увеличивает продажи на 15-20% и снижает возвраты на 25-30%.

[29] Составлено автором

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы я разработала информационную систему для подбора косметических средств. Все задачи, которые ставила в начале, выполнены в полном объёме.

По результатам анализа предметной области я выяснила, что существующие решения не обеспечивают полноценной персонализации – это подтвердило необходимость разработки собственной системы.

В проектной части я спроектировала архитектуру приложения на «ASP.NET Core MVC» с базой данных на «PostgreSQL», разработала логическую и физическую модель базы данных из 11 таблиц, а также создала алгоритм подбора с весовыми коэффициентами. Алгоритм учитывает 16 факторов одновременно: тип кожи, возраст, бюджет, проблемы, аллергии, сезон, любимые бренды, цель ухода, уровень стресса, тип питания, чувствительность к солнцу, склонность к отёкам, профессиональный уход, текстуру, готовность к многоступенчатому уходу и бонусы. Каждый фактор обрабатывается отдельным методом – всего таких методов 17 [4, 13].

В ходе разработки реализовала все ключевые функции: регистрацию и авторизацию с хешированием паролей, анкету из 15 шагов, алгоритм подбора, личный кабинет с историей подборок и избранным, корзину с «AJAX-добавлением», оформление заказа с промокодами и подарочными картами, а также админ-панель для управления каталогом. Система поддерживает «PWA»: её можно установить на телефон и пользоваться как полноценным мобильным приложением.

Тестирование провела на пяти сценариях, охватывающих разные типы кожи, возрастные группы и проблемы. Во всех случаях система выдала логичные результаты: товары, не подходящие по типу кожи, аллергенам или сезону, не попали в ТОП-10.

Экономическая эффективность подтверждена расчётами. Пользователь экономит до 30% бюджета (при бюджете 5000 руб/мес это около 1500 руб/мес) и сокращает время подбора с 3-4 часов до 5 минут. Магазин получает рост

продаж на 15-20% и снижение возвратов на 25-30%. Проект реализован без финансовых затрат на лицензии – все инструменты бесплатные.

Практическая значимость работы состоит в том, что система готова к использованию и может быть внедрена без серьёзных затрат. Она помогает пользователю принимать осознанные решения, а магазину – повышать доверие клиентов.

В качестве направлений дальнейшего развития можно выделить расширение алгоритма за счёт учёта отзывов и оценок пользователей, интеграцию с реальными интернет-магазинами для оформления заказа прямо из системы, а также добавление рекомендаций на основе истории покупок и сезонных изменений кожи.

Таким образом, все поставленные задачи выполнены, цель курсовой работы достигнута.

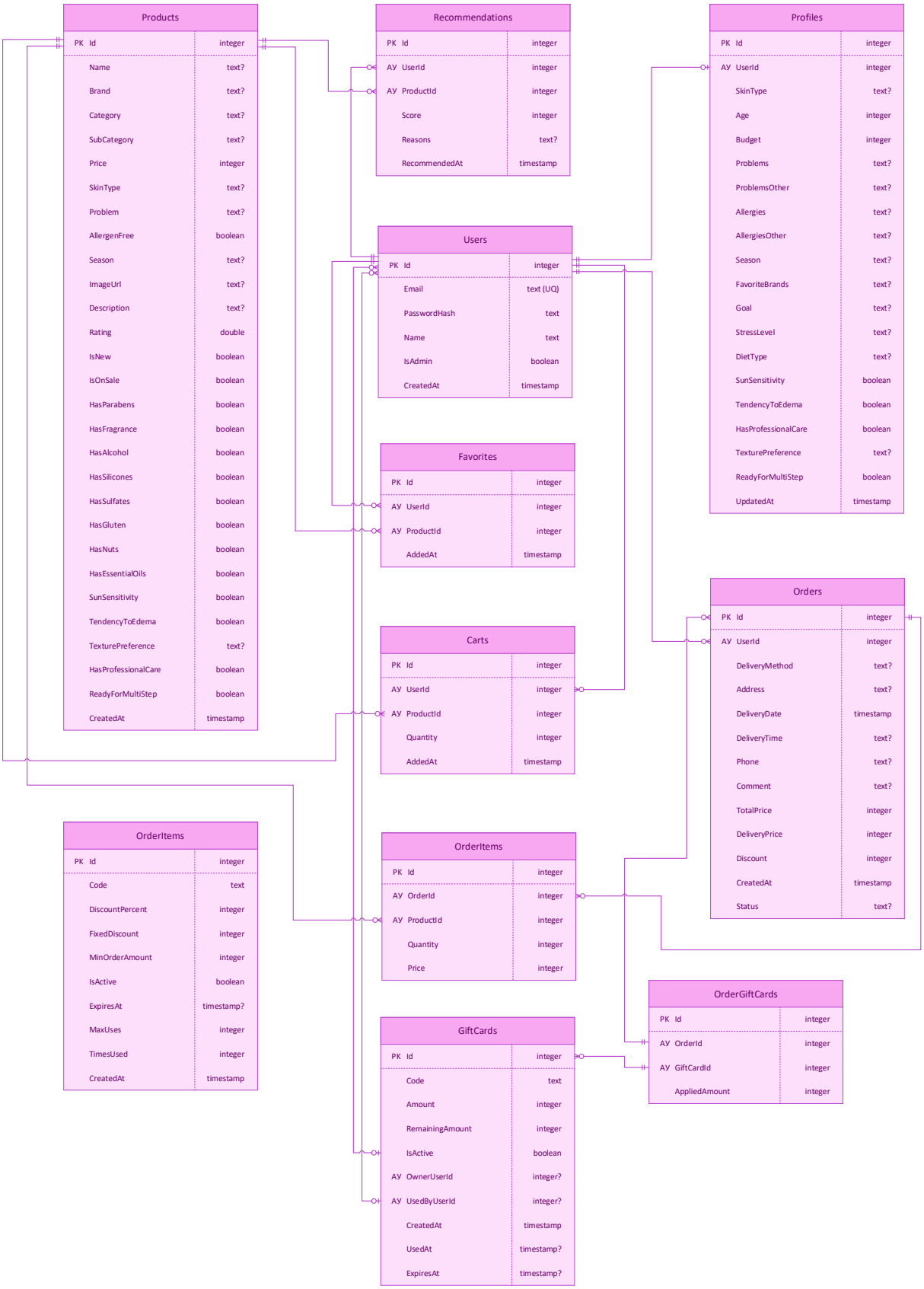
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Выручка российских производителей косметики в 2025 году превысила 1,16 трлн руб., на их долю пришлось до 80% оборота // Финмаркет. – 2026. – 3 мая. – URL: https://www.finmarket.ru/news/6613595.
2. Гусенко М., Российская крем-революция: как отечественная косметика захватила рынок // Российская газета. – 2026. – 3 мая. – URL: https://rg.ru/2026/05/03/rossijskaia-krem-revoliuciia-kak-otechestvennaia-kosmetika-zahvatila-80-rynka.html.
3. Морозова Я., Россияне начали экономить на косметике // Retail.ru. – 2026. – 26 февраля. – URL: https://www.retail.ru/news/rossiyane-nachali-ekonomit-na-kosmetike/.
4. A Novel Approach for Smart Skincare Solution: Derma AI Recommendation System // IEEE Xplore. – 2025. – DOI: 10.1109/ETCC65847.2025.11108513.
5. A Review on Cosmetic Product Recommendation Using Deep Learning // IEEE Xplore. – 2024. – DOI: 10.1109/ICSCSA64454.2024.00009.
6. BCrypt.Net-Next // NuGet Gallery. – URL: https://www.nuget.org/packages/BCrypt.Net-Next/.
7. Deep Learning-Based Hybrid Recommendation System for Safe Approach to Curated Cosmetics // IEEE Xplore. – 2025. – DOI: 10.1109/IDCIOT64235.2025.10915032.
8. Generative AI-based Explainable Personalized Hybrid Skincare Recommendation System // Information Processing Society of Korea Transactions on Software and Data Engineering. – 2025. – Vol. 14, no. 9. – P. 695–703. – DOI: 10.3745/TKIPS.2025.14.9.695. – URL: https://www.kci.go.kr/kciportal/landing/article.kci?arti_id=ART003251754.
9. Microsoft. C# documentation // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/dotnet/csharp/.
10. Microsoft. Entity Framework Core для начинающих // Microsoft Learn. – URL: https://learn.microsoft.com/ru-ru/ef/core/.

11. Microsoft. Get started developing a PWA // Microsoft Edge Developer documentation. – URL: https://learn.microsoft.com/en-us/microsoft-edge/progressive-web-apps-chromium/how-to/.
12. PostgreSQL Global Development Group. PostgreSQL 16 Documentation // PostgreSQL. – URL: https://www.postgresql.org/docs/16/index.html.
13. Skincare Recommendation System based on User's Skin Characteristics Similarity using Collaborative Filtering Approach // IEEE Xplore. – 2024. – DOI: 10.1109/ICEECIT63698.2024.10859798.

ПРИЛОЖЕНИЕ А

«ER-диаграмма» базы данных



ПРИЛОЖЕНИЕ Б

«SQL-скрипт» создания таблицы «Products»

```
CREATE TABLE "Products" (
    "Id" SERIAL PRIMARY KEY,
    "Name" VARCHAR(200) NOT NULL,
    "Brand" VARCHAR(100) NOT NULL,
    "Category" VARCHAR(100),
    "SubCategory" VARCHAR(100),
    "Price" DECIMAL(10,2) NOT NULL,
    "SkinType" VARCHAR(50),
    "Problem" VARCHAR(100),
    "Season" VARCHAR(50),
    "ImageUrl" TEXT,
    "Description" TEXT,
    "Rating" DECIMAL(3,1),
    "IsNew" BOOLEAN DEFAULT FALSE,
    "IsOnSale" BOOLEAN DEFAULT FALSE,
    "HasParabens" BOOLEAN DEFAULT FALSE,
    "HasFragrance" BOOLEAN DEFAULT FALSE,
    "HasAlcohol" BOOLEAN DEFAULT FALSE,
    "HasSilicones" BOOLEAN DEFAULT FALSE,
    "HasSulfates" BOOLEAN DEFAULT FALSE,
    "HasGluten" BOOLEAN DEFAULT FALSE,
    "HasNuts" BOOLEAN DEFAULT FALSE,
    "HasEssentialOils" BOOLEAN DEFAULT FALSE,
    "SunSensitivity" BOOLEAN DEFAULT FALSE,
    "TendencyToEdema" BOOLEAN DEFAULT FALSE,
    "TexturePreference" VARCHAR(50),
    "HasProfessionalCare" BOOLEAN DEFAULT FALSE,
    "ReadyForMultiStep" BOOLEAN DEFAULT FALSE
);
```

ПРИЛОЖЕНИЕ В

Фрагмент метода «CheckAllergies()» обработки аллергий

```
var allergenMap = new Dictionary<string, Func<Product, bool>>
{
    { "Парабены", p => p.HasParabens },
    { "Отдушки", p => p.HasFragrance },
    { "Спирт", p => p.HasAlcohol },
    { "Силиконы", p => p.HasSilicones },
    // ...

};

foreach (var allergy in userAllergies)
{
    if (allergenMap.ContainsKey(allergy) &&
allergenMap[allergy](product))
    {
        res.Score -= 50;
        warnings.Add($"содержит «{allergy}» — аллергия!");
    }
    else
    {
        res.Score += 10;
    }
}
```

ПРИЛОЖЕНИЕ Г

Фрагмент класса «ApplicationDbContext»

```
using Microsoft.EntityFrameworkCore;
using LuxDustApp.Models;
namespace LuxDustApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
        public DbSet<Profile> Profiles { get; set; }
        public DbSet<Product> Products { get; set; }
        public DbSet<Recommendation> Recommendations { get; set;
}
        public DbSet<Favorite> Favorites { get; set; }
        public DbSet<Cart> Carts { get; set; }
        public DbSet<PromoCode> PromoCodes { get; set; }
        public DbSet<GiftCard> GiftCards { get; set; }
        public DbSet<OrderGiftCard> OrderGiftCards { get; set; }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderItem> OrderItems { get; set; }
    }
}
```

ПРИЛОЖЕНИЕ Д

Фрагмент метода «GetRecommendationsWithReasons()»

```
public RecommendationBundle
GetRecommendationsWithReasons(Profile profile)
{
    var allProducts = _context.Products.ToList();
    var result = new Dictionary<Product,
RecommendationResult>();
    var bundle = new RecommendationBundle();
    if (allProducts == null || !allProducts.Any())
        return bundle;

    var excludedCategories = new List<string> { "Подборки",
"Уборка и стирка", "Хобби и творчество", "Фигура мечты" };

    bool userWantsFaceCare = !string.IsNullOrEmpty(profile.Goal)
&&
        RecommendationConstants.FaceCareGoals.Any(g =>
profile.Goal.Contains(g));

    var userAllergies = (profile.Allergies ?? "")
        .Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(a => a.Trim())
        .Where(a => a != "Нет")
        .ToList();

    foreach (var product in allProducts)
    {
        if (!string.IsNullOrEmpty(product.Category) &&
excludedCategories.Contains(product.Category))
            continue;

        if (userWantsFaceCare &&
!string.IsNullOrEmpty(product.Category) &&

RecommendationConstants.NonFaceCareCategories.Contains(product.C
ategory))
            continue;

        var res = new RecommendationResult();
        var reasons = new List<string>();
        var warnings = new List<string>();

        CheckSkinType(profile, product, res, reasons, warnings);
        CheckAge(profile, product, res, reasons, warnings);
        CheckBudget(profile, product, res, reasons, warnings);
        CheckProblems(profile, product, res, reasons, warnings);
        CheckAllergies(profile, product, userAllergies, res,
reasons, warnings);
        CheckSeason(profile, product, res, reasons, warnings);
        CheckBrands(profile, product, res, reasons, warnings);
```

```
        CheckGoal(profile, product, res, reasons, warnings);
        CheckStress(profile, product, res, reasons, warnings);
        CheckDiet(profile, product, res, reasons, warnings);
        CheckSunSensitivity(profile, product, res, reasons,
warnings);
        CheckEdema(profile, product, res, reasons, warnings);
        CheckProfessionalCare(profile, product, res, reasons,
warnings);
        CheckTexture(profile, product, res, reasons, warnings);
        CheckMultiStep(profile, product, res, reasons,
warnings);
        CheckCategory(profile, product, res, reasons, warnings,
userWantsFaceCare);
        CheckBonuses(product, res, reasons);

        res.Reasons = reasons.Any() ? " ✓ " + string.Join("; ",
reasons) + "." : "Нет явных совпадений.";
        if (warnings.Any())
            res.Reasons += " ! " + string.Join("; ", warnings) +
".";

        result[product] = res;
    }

    var sorted = result.OrderByDescending(kv =>
kv.Value.Score).ToList();

    bundle.Top = sorted.Take(10).Select(kv => kv.Key).ToList();
    bundle.TopReasons = sorted.Take(10).ToDictionary(kv =>
kv.Key, kv => kv.Value);

    bundle.Middle = sorted.Skip(10).Take(50).Select(kv =>
kv.Key).ToList();
    bundle.MiddleReasons =
sorted.Skip(10).Take(50).ToDictionary(kv => kv.Key, kv =>
kv.Value);

    return bundle;
}
```

ПРИЛОЖЕНИЕ Е

Фрагмент метода «Results()» контроллера «QuizController»

```
[HttpPost]
public IActionResult Results(Profile profile)
{
    var userId = GetUserId();
    if (userId == null) return RedirectToAction("Login",
"Account");

    if (string.IsNullOrEmpty(profile.SkinType) || profile.Age <=
0 || profile.Budget <= 0)
    {
        ModelState.AddModelError("", "Пожалуйста, заполните все
обязательные поля анкеты.");
        return View("Step1", profile);
    }

    var bundle =
_recommendationService.GetRecommendationsWithReasons(profile);

    foreach (var kvp in bundle.TopReasons)
    {
        _context.Recommendations.Add(new Recommendation
        {
            UserId = userId.Value,
            ProductId = kvp.Key.Id,
            Score = kvp.Value.Score,
            Reasons = kvp.Value.Reasons,
            RecommendedAt = System.DateTime.UtcNow
        });
    }
    _context.SaveChanges();

    var existingProfile = _context.Profiles.FirstOrDefault(p =>
p.UserId == userId.Value);
    if (existingProfile == null)
    {
        profile.UserId = userId.Value;
        profile.UpdatedAt = System.DateTime.UtcNow;
        _context.Profiles.Add(profile);
    }
    else
    {
        existingProfile.SkinType = profile.SkinType;
        existingProfile.Age = profile.Age;
        existingProfile.Budget = profile.Budget;
        existingProfile.Problems = profile.Problems;
        existingProfile.Allergies = profile.Allergies;
        existingProfile.Season = profile.Season;
        existingProfile.FavoriteBrands = profile.FavoriteBrands;
        existingProfile.Goal = profile.Goal;
```

```
        existingProfile.StressLevel = profile.StressLevel;
        existingProfile.DietType = profile.DietType;
        existingProfile.SunSensitivity = profile.SunSensitivity;
        existingProfile.TendencyToEdema =
profile.TendencyToEdema;
        existingProfile.HasProfessionalCare =
profile.HasProfessionalCare;
        existingProfile.TexturePreference =
profile.TexturePreference;
        existingProfile.ReadyForMultiStep =
profile.ReadyForMultiStep;
        existingProfile.UpdatedAt = System.DateTime.UtcNow;
    }
    _context.SaveChanges();

    return View(bundle);
}
```